Construcción y prueba de penetración de una tienda en línea con *vibe coding*: tutorial del caso Byte

Neyder Sebastian Orozco Villamil

Ingeniería de Sistemas, Facultad de Ingeniería y Ciencias Básicas, Universidad Central

43391427: Hacking Ético

Mg. Carlos Ivan Pinzon Romero

13 de noviembre de 2026

Resumen

Este documento es un tutorial, basado en un caso real, sobre cómo construir una aplicación web de producción mediante *vibe coding*, es decir, programando a través de instrucciones en lenguaje natural a un agente de inteligencia artificial. El caso es Byte, una tienda en línea y punto de venta de videojuegos, consolas y accesorios para Colombia, construida con Django 5.2, PostgreSQL 17, Celery y un módulo de inteligencia de negocio, y desplegada en Render. Toda la construcción partió de un único prompt maestro que fijó el rol de la inteligencia artificial, las reglas de trabajo, la arquitectura, los requisitos funcionales y de seguridad, el plan de diez fases y los criterios de aceptación. El tutorial explica ese prompt y recorre las fases con el fragmento que gobernó cada una, su implementación y sus evidencias. La aplicación terminó con 727 pruebas, una cobertura del 93 %, cero hallazgos de semgrep y cero fallos en OWASP ZAP baseline. La segunda parte es el informe de la prueba de penetración hecha desde Kali Linux contra la instancia desplegada: el control de acceso y las defensas contra inyección resistieron, y se encontraron dos hallazgos de severidad media en la autenticación, la protección contra fuerza bruta inefectiva detrás del *proxy* y un secreto TOTP público en las cuentas demo del personal, que las verificaciones automáticas no habían detectado y que se corrigieron y re-probaron.

Palabras clave: vibe coding, prompt maestro, desarrollo asistido por inteligencia artificial, Django, seguridad de aplicaciones, OWASP Top 10, pruebas de penetración, Kali Linux

Construcción y prueba de penetración de una tienda en línea con *vibe coding*: tutorial del caso Byte

Construir una tienda en línea completa (catálogo, carrito, pagos, panel administrativo, inventario multisede, punto de venta y analítica) suele requerir un equipo y varios meses. Este tutorial muestra cómo se construyó una aplicación de ese tamaño con *vibe coding*: el autor no escribió el código línea a línea, sino que lo especificó en un prompt maestro y supervisó a un agente de inteligencia artificial (IA) que trabaja directamente sobre el repositorio, ejecuta comandos y verifica sus propios resultados.

El trabajo se hizo para el curso de Hacking ético. Por eso la seguridad no fue un añadido final: fue un requisito del prompt maestro desde la primera línea, alineado con el OWASP Top 10:2025 (OWASP Foundation, 2025). La aplicación resultante, llamada Byte, fue después sometida a una prueba de penetración desde Kali Linux, que se documenta en la segunda parte.

El documento tiene tres propósitos. El primero es enseñar un método replicable: cómo redactar un prompt maestro, qué preparar antes de enviarlo y cómo verificar lo que la IA entrega en cada fase. El segundo es dejar constancia de lo que ocurrió en este caso, con los fragmentos del prompt que guiaron cada fase y las evidencias que se recogieron, incluidos los errores y su corrección. El tercero es mostrar qué encontró un atacante en la aplicación terminada.

El tutorial se organiza así. Primero se presentan el marco de referencia y el caso. Luego se describen la preparación del entorno, la anatomía del prompt maestro, la aprobación del plan y las reglas persistentes del proyecto. Después se recorre la construcción fase por fase y se resumen los resultados, las decisiones que tomó la IA donde el prompt dejaba opciones y las lecciones aprendidas. La segunda parte es el informe de la prueba de penetración, con su alcance, sus fases, los hallazgos y el plan de remediación. El Apéndice A contiene el prompt maestro completo y el Apéndice B, un índice de las evidencias.

Marco de referencia

Qué es el *vibe coding*

El término fue propuesto por Karpathy (2025) para describir una forma de programar en la que la persona se entrega a la conversación con un modelo de lenguaje, acepta sus cambios y casi olvida que el código existe. En su forma más pura, el *vibe coding* sirve para prototipos rápidos: se pide algo, se prueba si funciona y se pide el siguiente cambio, sin leer en detalle lo que la IA escribió.

Para una aplicación que maneja dinero, datos personales e inventario, esa versión pura no es suficiente. Una IA puede producir código que funciona en la demostración y aun así tiene fallas de control de acceso, secretos expuestos o dependencias vulnerables. Por eso, en este proyecto se usó una variante guiada por especificación: la persona no escribe el código, pero define con precisión qué debe construirse, cómo debe trabajar la IA y cómo se demuestra que el resultado funciona.

El prompt maestro como especificación

El eje del método es un prompt maestro: un único mensaje extenso que cumple el papel de un documento de requisitos y, a la vez, de un contrato de trabajo con la IA. A diferencia de una cadena de pedidos sueltos, el prompt maestro fija desde el inicio cuatro cosas:

1. **El rol y las reglas de la IA**: con qué experiencia debe actuar y qué debe hacer siempre (probar, analizar, documentar, hacer commits, detenerse).

2. **El alcance**: tecnologías, arquitectura, roles, módulos y requisitos de seguridad, con un nivel de detalle verificable.

3. **El plan**: diez fases con su entregable y la obligación de detenerse al final de cada una para la revisión humana.

4. **La definición de terminado**: criterios de aceptación que se pueden comprobar uno a uno.

El agente usado fue Claude Code (Anthropic, s. f.), en su aplicación de escritorio para Windows. Este tipo de agente no solo genera texto: lee y edita archivos del proyecto, ejecuta comandos en la terminal, levanta contenedores, corre las pruebas y opera un navegador para tomar capturas. Esa capacidad de ejecutar y verificar es la que permite que un prompt maestro se convierta en una aplicación funcionando.

El caso: Byte

Byte es la tienda en línea y punto de venta de una cadena ficticia de videojuegos, consolas y accesorios en Colombia. Maneja precios en pesos colombianos con IVA del 19 % incluido, envíos por departamento y ciudad, y obligaciones legales locales: el Estatuto del Consumidor (Congreso de Colombia, 2011), que prohíbe anunciar descuentos sobre precios que no se cobraron y fija el derecho de retracto, y la ley de protección de datos personales (Congreso de Colombia, 2012). La Tabla 1 resume sus módulos.

Tabla 1

Módulos de la aplicación Byte

Módulo	Funciones principales
Tienda	Catálogo con variantes configurables y precio dinámico, búsqueda tolerante a errores, comparador, carrito de invitado, checkout con cupones y envío, pagos en modo de pruebas, factura en PDF.
Cuenta del cliente	Pedidos, devoluciones, retomas de consolas usadas, servicio técnico, puntos y saldo a favor, alertas de precio y disponibilidad, privacidad (descarga y eliminación de datos).
Panel (`/panel/`)	Resumen por rol, pedidos, inventario multisede con kardex, ajustes y traslados, compras, punto de venta con arqueo, clientes, precios, cupones, reportes, auditoría y eventos de seguridad.
Inteligencia de negocio	Histórico sintético, flujo de datos bronce, plata y oro, modelo estrella, segmentación RFM, pronóstico de ventas y tablero interactivo.
API (`/api/v1/`)	API versionada que deniega por defecto; esquema OpenAPI solo para el personal.

Nota. Elaboración propia a partir del prompt maestro y del código del repositorio.

Preparación del entorno

Antes de enviar el prompt maestro conviene tener listo el entorno, porque la IA ejecuta comandos reales y se detiene cuando falta una herramienta. La Tabla 2 lista lo que se usó.

Tabla 2

Herramientas del entorno de construcción

Herramienta	Uso en el proyecto
Claude Code (aplicación de escritorio)	Agente de IA que edita el repositorio, ejecuta comandos, opera el navegador y toma capturas.
Windows 11 + Python 3.12	Sistema operativo y lenguaje base.
uv (Astral, s. f.)	Gestor de dependencias con archivo de bloqueo, *hashes* y cuarentena de 7 días para paquetes recién publicados.
Docker Desktop + WSL2	Contenedores para PostgreSQL, Valkey, el worker de Celery y la imagen de producción.
Git y GitHub	Control de versiones con una rama por fase y repositorio remoto para el despliegue.
Playwright (Microsoft, s. f.)	Capturas de pantalla automáticas y prueba de extremo a extremo de la compra.
semgrep, bandit, pip-audit, Trivy y ZAP (Semgrep, Inc., s. f.; ZAP Development Team, s. f.)	Análisis estático, auditoría de dependencias, escaneo de la imagen y escaneo dinámico.
Render (Render, s. f.)	Plataforma de despliegue definida como código.

Nota. Las versiones exactas de las librerías se verificaron en PyPI y npm el día en que se fijaron y quedaron registradas en el ADR-0005 y en el archivo `uv.lock`.

Lección para replicar

Instale y pruebe la infraestructura (Docker con WSL2, base de datos, Git) antes de enviar el prompt. En este caso Docker no estaba instalado: la IA lo detectó al intentar correr las pruebas contra PostgreSQL, y su instalación reinició el equipo a mitad de la Fase 1. Lo que ya estaba confirmado en Git sobrevivió; lo que solo estaba en la conversación hubo que reconstruirlo.

Paso 1: redactar el prompt maestro

El paso más importante del método es la especificación. En este caso fue un prompt de unos 28 000 caracteres, organizado en trece secciones numeradas del 0 al 12 (Tabla 3). El texto completo está en el Apéndice A.

Tabla 3

Estructura del prompt maestro

Sección	Para qué sirve
0. Rol, contexto y reglas	Define el papel de la IA (arquitecto y desarrollador sénior con experiencia en seguridad) y las reglas no negociables: fases, commits, secretos, idioma, pruebas.
1. Stack tecnológico	Fija las tecnologías por capa para que la IA no improvise.
2. Arquitectura	Monolito modular con capa de servicios, lista de apps, identificadores públicos UUID y dinero en decimales.
3. Usuarios y roles	Diez roles, denegación por defecto, permisos por objeto, separación de funciones y matriz de permisos generada desde el código.
4–6. Tienda, panel y analítica	Requisitos funcionales de cada módulo, con el nivel de detalle de un documento de requisitos.
7. Seguridad OWASP 2025	Controles obligatorios por cada categoría del Top 10, protección de formularios y suite de pruebas de seguridad.
8–10. API, calidad y despliegue	Reglas de la API, metas de calidad y cobertura, e infraestructura como código.
11. Plan de fases	Diez fases con su entregable; la IA debe detenerse al final de cada una.
12. Criterios de aceptación	Lista verificable de lo que debe cumplirse al terminar.

Nota. Elaboración propia a partir del prompt maestro (Apéndice A).

Sección 0: el rol y las reglas

La sección 0 es la que gobierna el comportamiento de la IA durante toda la obra (Fragmento 1). Cada regla responde a un riesgo concreto del desarrollo con IA: perder el control del avance (fases y commits), olvidar las convenciones entre sesiones (`CLAUDE.md`), filtrar secretos (variables de entorno), decidir de forma opaca (ADR), usar versiones inventadas u obsoletas (verificación de versiones) y romper la lógica de negocio sin notarlo (pruebas obligatorias).

Fragmento 1 del prompt maestro. Rol, contexto y reglas de trabajo

Actúa como arquitecto de software senior y desarrollador full-stack Django, con experiencia en e-commerce, ingeniería de datos y seguridad de aplicaciones (AppSec). Vas a construir desde cero una aplicación web de producción para Byte, una tienda de videojuegos, consolas y accesorios en Colombia (moneda COP, IVA 19 %, envíos nacionales por departamento/ciudad).

Reglas de trabajo obligatorias:

Trabaja por fases (sección 11). Al final de cada fase: ejecuta los tests, el linter y el análisis de seguridad, actualiza CHANGELOG.md, haz un commit con mensaje convencional (feat:, fix:, sec:,

docs:…) y detente para que yo revise.

Antes de escribir código, crea CLAUDE.md con las convenciones del proyecto, los comandos (correr, testear, migrar, sembrar datos, desplegar) y las decisiones de arquitectura. Mantenlo actualizado.

Nunca pongas secretos en el código ni en el repo. Todo va por variables de entorno (.env.example documentado, .env en .gitignore).

Si una decisión tiene varias opciones razonables, elige la más simple y segura, y documéntala como ADR corto en docs/adr/.

Si una librería o servicio no está disponible o cambió, verifica la versión actual estable antes de fijarla y anota el cambio.

Código, nombres de modelos y comentarios en inglés; interfaz de usuario, mensajes y documentación en español.

Todo cambio de lógica de negocio importante debe tener tests.

Secciones 1 a 10: el alcance, escrito para ser verificable

Las secciones de alcance comparten un rasgo: cada requisito se puede comprobar. El prompt no dice «la aplicación debe ser segura», sino qué control se espera y cómo se demuestra. Los requisitos de control de acceso del Fragmento 2 son un buen ejemplo: cada línea se tradujo después en código y en pruebas automáticas.

Fragmento 2 del prompt maestro. Requisitos de control de acceso (sección 3.2)

Denegar por defecto: toda vista del panel exige autenticación + permiso explícito. Crea mixins/decoradores propios (RoleRequiredMixin, ObjectPermissionMixin) y úsalos en TODAS las vistas y endpoints.

Permisos a nivel de objeto: un cliente solo ve sus pedidos, direcciones, tickets y retomas; un gerente solo ve las sedes asignadas.

Separación de funciones: quien crea un ajuste de inventario o un descuento > [X] % no puede aprobarlo.

[…]

Tests que prueban, para cada endpoint sensible, que los roles sin permiso reciben 403/404 (incluye pruebas de IDOR cambiando UUIDs).

El prompt también deja opciones abiertas a propósito: el umbral `[X]` % del descuento, la plataforma de despliegue `[PLATAFORMA_DE_DESPLIEGUE]` y varias parejas de tecnologías («django-guardian o servicio propio», «Wompi o Mercado Pago»). Esas opciones se resuelven con la regla de la sección 0: elegir la más simple y segura y documentarla como ADR. La Tabla 9, más adelante, muestra cómo se resolvió cada una.

Secciones 11 y 12: el plan y la definición de terminado

El plan de fases (Fragmento 3) divide la obra en entregables revisables, y los criterios de aceptación permiten saber, al final, qué se cumplió y qué no.

Fragmento 3 del prompt maestro. Plan de fases (sección 11)

1. Fundaciones	Estructura, settings por entorno, Docker Compose, CI básico, CLAUDE.md, ADRs iniciales, modelo de datos y diagrama ER
2. Usuarios y seguridad base	Auth con allauth + MFA, axes, roles y permisos, mixins, auditoría, cabeceras/CSP, tests de acceso
3. Catálogo y precios	Productos, variantes y configuraciones con precio dinámico, búsqueda, filtros, comparador, historial de precios
4. Carrito, checkout y pagos	Carrito, cupones, envíos, pasarela sandbox, webhooks firmados, pedidos, facturas PDF, correos
5. Panel administrativo	Dashboard, inventario multi-sede y kardex, compras, pedidos, POS, CRM, moderación, reportes
6. Módulos de valor agregado	Retomas, servicio técnico, fidelidad, preventas, alertas de precio/stock, devoluciones
7. Big Data / BI	Generador de datos, pipeline medallón, modelo estrella, modelos analíticos, dashboards con insights, notebook PySpark
8. Endurecimiento de seguridad	Matriz OWASP completa, modelo de amenazas, suite de seguridad, ZAP, SBOM, revisión de hallazgos
9. Despliegue	Infraestructura como código, CI/CD, despliegue, datos demo, verificación end-to-end
10. Documentación final	README, manual de usuario por rol (con capturas), manual técnico, docs/seguridad.md, docs/big-data.md, guion de sustentación

El título de la sección 11 repite la instrucción clave: «detente al final de cada una». Junto con la regla de la sección 0 («detente para que yo revise»), convierte cada fase en una puerta de aprobación: la IA no avanza a la siguiente hasta que la persona revisa la anterior.

Lección para replicar

Al redactar su propio prompt maestro: (1) dé a la IA un rol con experiencia en seguridad; (2) escriba reglas de trabajo que obliguen a probar, documentar y detenerse; (3) formule cada requisito de forma que se pueda comprobar; (4) marque con corchetes los valores que deban decidirse y pida que cada decisión quede en un ADR; (5) cierre con criterios de aceptación que permitan saber cuándo está terminado.

Paso 2: revisar y aprobar el plan

Antes de escribir código conviene pedir a la IA el plan de la primera fase y revisarlo. En este caso la IA presentó tres entregables: el árbol de carpetas, el modelo de datos en Mermaid que pedía la sección 2 y la lista de dependencias. Cada dependencia venía con la versión estable verificada ese día en PyPI y npm, como exigía la sección 0. Por ejemplo, propuso Django 5.2.19, la versión LTS, en lugar de la 6.1.3, que era la más reciente, porque la sección 1 pedía «Django 5.2 LTS (o la LTS vigente)» y la LTS tiene soporte durante todo el proyecto (Django Software Foundation, 2025). Tras revisar el plan, la persona lo aprobó y comenzó la Fase 1.

Paso 3: convertir las reglas en archivos del repositorio

Una conversación con la IA se pierde cuando se reinicia la sesión o cuando el contexto se resume. Por eso la sección 0 pide crear `CLAUDE.md` antes de escribir código. En este caso las reglas quedaron en tres lugares:

1. **`CLAUDE.md`**: convenciones, comandos y reglas de trabajo que el agente lee al iniciar cada sesión. Se actualizó en el mismo commit cada vez que cambió una convención.
2. **Registros de decisiones de arquitectura** (ADR) en `docs/adr/`: diecisiete decisiones cortas con su contexto, alternativas y consecuencias.
3. **La bitácora** en `docs/bitacora/`: un archivo por fase con las secciones *Qué, Cómo, Por qué, Evidencias* y *Pendientes*, acompañado de capturas en `docs/capturas/`.

El fragmento siguiente, tomado de `CLAUDE.md`, muestra cómo las reglas del prompt maestro quedaron convertidas en instrucciones permanentes.

```
## Reglas de trabajo (obligatorias)

1. **Fases**: se trabaja por las fases de la tabla de abajo. Al cerrar cada fase
   se sigue este orden: tests, linters y análisis de seguridad → CHANGELOG.md →
   bitácora con capturas → commit convencional → **detenerse para revisión**.
2. **Ramas**: cada fase vive en `fase/NN-nombre` y se fusiona a `main` solo
   cuando el usuario la aprueba.
4. **Secretos**: nunca en el código ni en el repositorio. Todo va por variables
```

```
   de entorno (`.env.example` documentado y `.env` ignorado por git).
5. **Decisiones**: cuando haya varias opciones razonables, se elige la más
   simple y segura y se documenta como ADR en `docs/adr/`.
```

La regla «ejecuta los tests, el linter y el análisis de seguridad» quedó automatizada en `scripts/evidencia.py`, que corre todas las verificaciones y guarda su salida como texto y como imagen de terminal. Un segundo script, `scripts/capturas.py`, abre la aplicación con Playwright y toma capturas en escritorio y móvil, en modo claro y oscuro, revisando además la consola del navegador en busca de violaciones de la política de seguridad de contenido (CSP). La Tabla 4 resume las verificaciones, que son las mismas que corre el pipeline de integración continua que pide la sección 7.

Tabla 4

Verificaciones automáticas ejecutadas al cerrar cada fase

N.º	Verificación	Qué detecta
01	pytest con cobertura	Regresiones funcionales y de seguridad, contra PostgreSQL real.
02–04	ruff, black, mypy	Errores de estilo, formato y tipos.
05	bandit	Patrones inseguros en Python (SAST).
06	pip-audit	Dependencias con vulnerabilidades conocidas.
07	`manage.py check --deploy`	Configuración insegura de Django en producción.
08	Cabeceras de `GET /`	CSP con *nonce*, HSTS, X-Frame-Options, COOP, CORP y Permissions-Policy.
09	Integridad de librerías JS	Que htmx, Alpine, ECharts y Tailwind coincidan con sus *hashes* publicados.
10	semgrep	Inyección, redirecciones abiertas, criptografía débil y otros patrones.
11	Trivy	Vulnerabilidades altas o críticas en la imagen Docker.
12	Matriz de permisos	Que la matriz rol × permiso documentada coincida con el código.
13	Cadena de auditoría	Que la bitácora de auditoría encadenada por *hash* esté íntegra.
14–15	SBOM y ZAP baseline	Inventario de componentes y escaneo dinámico pasivo (desde la Fase 8).

Nota. Elaboración propia a partir de `scripts/evidencia.py` y `.github/workflows/ci.yml`.

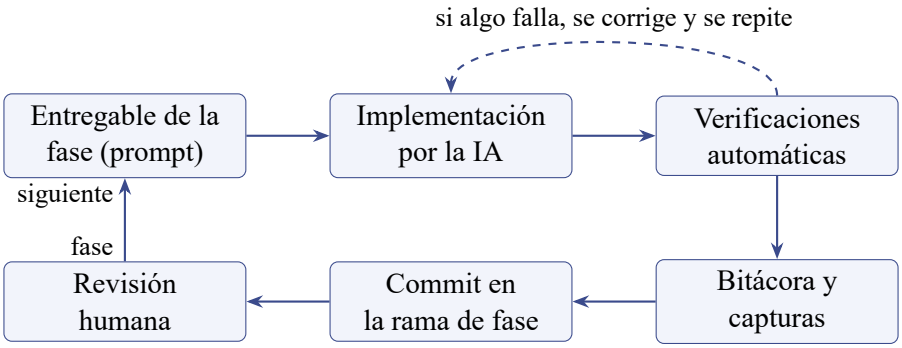
Paso 4: construir por fases

Con el plan aprobado y las reglas fijadas, la obra avanzó en las diez fases de la sección 11. Cada una siguió el mismo ciclo (Figura 1): el entregable definido en el prompt

maestro, la implementación por parte de la IA, las verificaciones automáticas, la bitácora con capturas, el commit en la rama de la fase y la revisión humana antes de pasar a la siguiente.

Figura 1

Ciclo de trabajo de cada fase



Nota. Elaboración propia a partir de la regla de trabajo de la sección 0 del prompt maestro.

La Tabla 5 resume el número de pruebas y la cobertura medidos al cerrar cada fase.

Tabla 5

Pruebas y cobertura al cerrar cada fase

N.º	Fase	Pruebas	Cobertura
1	Fundaciones	323	99,8 %
2	Usuarios y seguridad base	448	93,0 %
3	Catálogo y precios	530	94,6 %
4	Carrito, checkout y pagos	582	94 %
5	Panel administrativo	679	94 %
6	Módulos de valor agregado	699	93 %
7	Big data e inteligencia de negocio	713	93 %
8	Endurecimiento de seguridad	724	93 %
9	Despliegue	727	93 %
10	Documentación final	727	93 %

Nota. Datos tomados de las bitácoras `docs/bitacora/fase-NN.md`. La Fase 9 incluye las pruebas agregadas al corregir el despliegue; la Fase 10 no cambió código. La meta del prompt maestro era una cobertura de al menos 80 %.

Fase 1: fundaciones

Fragmento 4 del prompt maestro. Arquitectura (sección 2)

Monolito modular Django con capa de servicios (la lógica de negocio NO vive en las vistas) […]

Cada app: models.py, services.py (lógica), selectors.py (consultas), forms.py, views.py, permissions.py, tests/.

Identificadores públicos en URLs y API: UUID o slug, nunca el ID autoincremental.

Precios en DecimalField (o enteros en pesos), nunca float.

Genera el diagrama entidad-relación en Mermaid en docs/modelo-datos.md.

Cómo se implementó. La IA generó veintiuna apps de Django con la misma estructura en capas y una prueba (`tests/test_structure.py`) que impide que una app nueva rompa la convención. Configuró los ajustes por entorno, Docker multietapa con usuario sin privilegios, Docker Compose con web, worker, beat, PostgreSQL y Valkey, la integración continua, `CLAUDE.md`, los primeros ADR y el modelo de datos en Mermaid. La seguridad de base (CSP con *nonce*, cabeceras, cookies seguras, Argon2, Celery con JSON, errores genéricos con identificador de correlación) se configuró desde esta fase.

La sección 1 pedía un «look “gamer” moderno» y la sección 9, un catálogo realista. La página de inicio se construyó como la de una tienda en línea completa, con carrusel, beneficios, categorías con foto, productos destacados, retoma, servicio técnico y marcas (Figuras 2 y 3). Las fotos de producto se tomaron de Wikimedia Commons, solo con licencia libre verificable (Wikimedia Commons, s. f.): un script fija cada archivo por su *hash* SHA-256, quita el fondo de estudio, genera WebP sin metadatos y publica los créditos.

Figura 2

Página de inicio de Byte en escritorio, modo oscuro



Nota. Captura tomada con Playwright al cerrar la Fase 1 (`docs/capturas/fase-01/inicio-escritorio-oscuro.jpg`), recortada a la parte visible de la pantalla. Las fotos de producto provienen de Wikimedia Commons con licencia libre.

Figura 3

Página de inicio en un teléfono, modo claro



Nota. Captura de la Fase 1 (`docs/capturas/fase-01/inicio-movil-claro.jpg`), recortada.

Qué encontraron las verificaciones. Al probar la imagen real en contenedores aparecieron problemas que no se veían sin ellos. La IA los detectó con las verificaciones y los corrigió antes del commit (Tabla 6). La Figura 4 muestra el resumen de las once verificaciones de la fase.

Tabla 6

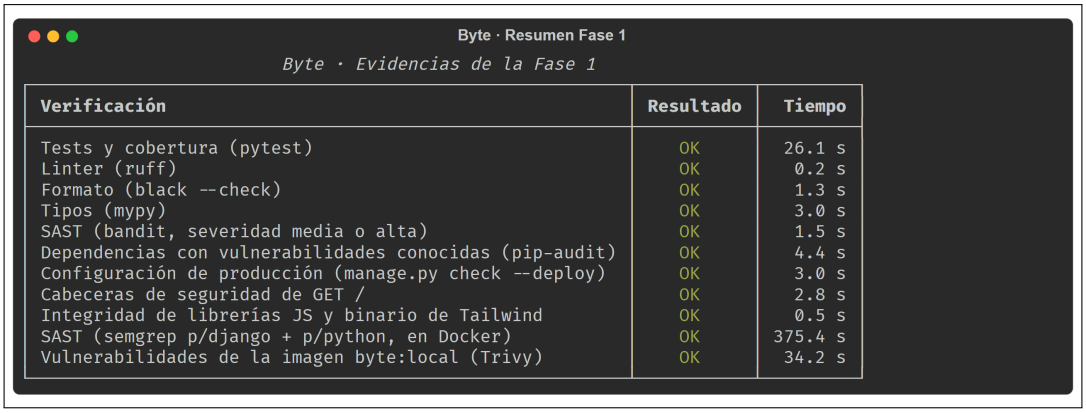
Hallazgos de la verificación con contenedores en la Fase 1

Hallazgo	Corrección
Los archivos estáticos quedaban ilegibles para el usuario sin privilegios: la tienda habría salido sin estilos ni imágenes.	Permisos propios para los estáticos y prueba de regresión.
La imagen de producción incluía las herramientas de seguridad (978 MB).	Exportar solo las dependencias de la aplicación: la imagen bajó a 472 MB.
semgrep señaló el uso de SHA-1 para verificar las fotos.	Cada foto se fija por SHA-256.
Trivy encontró dos CVE altas en `zlib1g` de la imagen base.	La etapa base aplica las actualizaciones de seguridad de Debian.
El worker de Celery no registraba la ejecución de tareas.	Se aplicó la configuración de registro de Django al worker.

Nota. Resumen de la bitácora `docs/bitacora/fase-01.md`.

Figura 4

Resumen de las verificaciones automáticas de la Fase 1



Nota. Imagen de terminal generada por `scripts/evidencia.py` (`docs/capturas/fase-01/00-resumen.svg`). Resultado: 323 pruebas aprobadas con cobertura del 99,83 % y cero hallazgos en los análisis de seguridad.

Fase 2: usuarios y seguridad base

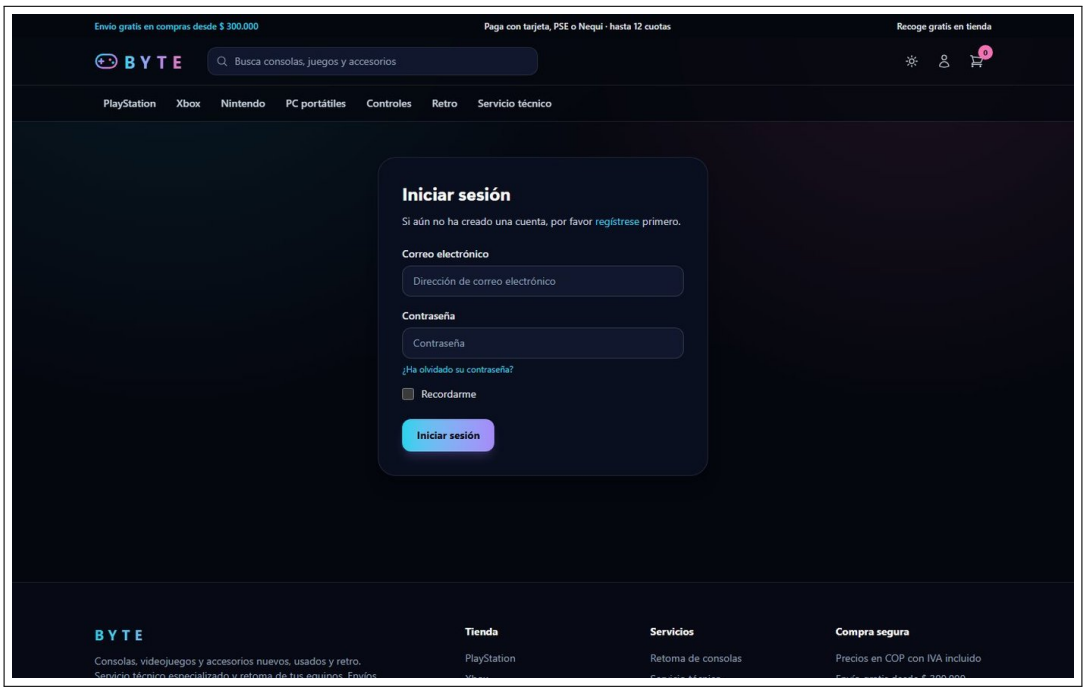
Fragmento 5 del prompt maestro. Fallas de autenticación (sección 7, A07:2025)

> django-allauth con verificación de correo, MFA TOTP (obligatorio para todo el personal), códigos de recuperación.
>
> django-axes: bloqueo progresivo por usuario+IP; CAPTCHA (Cloudflare Turnstile o hCaptcha) tras intentos fallidos y en registro.
>
> […]
>
> Mensajes de error genéricos (“credenciales inválidas”) para no enumerar usuarios.

Cómo se implementó. La IA usó allauth para registro, verificación, MFA y sesiones, y axes para el bloqueo progresivo por usuario e IP. Definió los roles en un solo archivo y generó desde el código la matriz de permisos, como pedía la sección 3.2. La auditoría quedó encadenada por *hash* y protegida con un disparador de PostgreSQL que impide modificar o borrar registros. El área «Mi cuenta» incluye el consentimiento, la descarga y la eliminación de datos que exige la ley de datos personales (Congreso de Colombia, 2012). La Figura 5 muestra el inicio de sesión y la Figura 6, la respuesta cuando un vendedor intenta entrar a la auditoría.

Figura 5

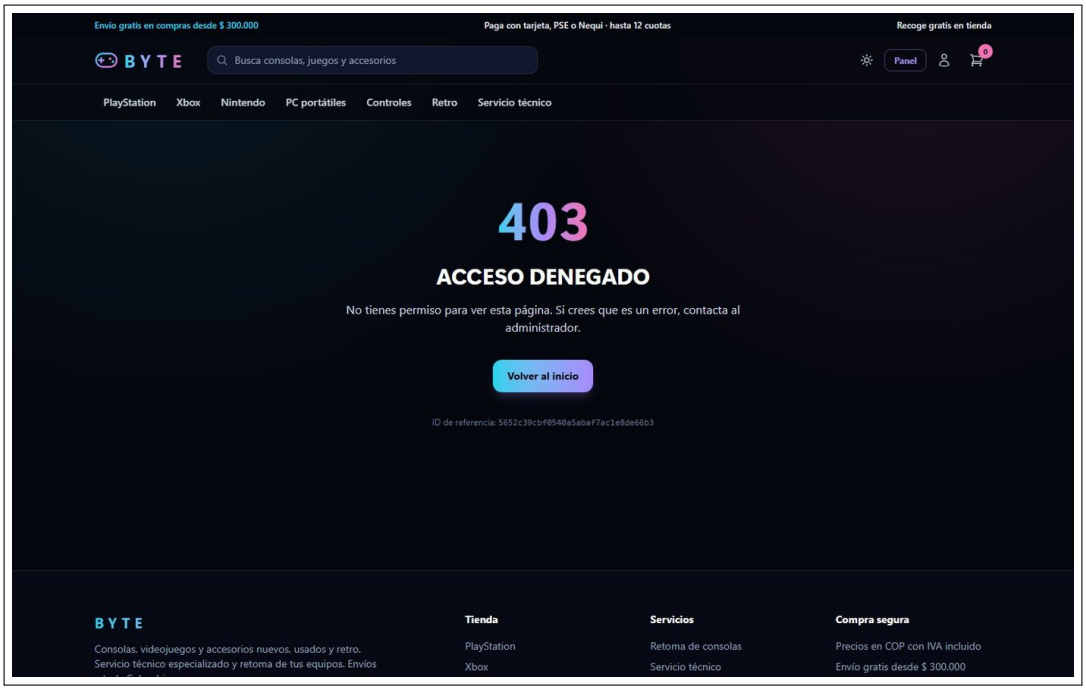
Inicio de sesión



Nota. Captura de la Fase 2 (`docs/capturas/fase-02/login-escritorio-oscuro.jpg`).

Figura 6

Un vendedor intenta abrir la auditoría y recibe «Acceso denegado»

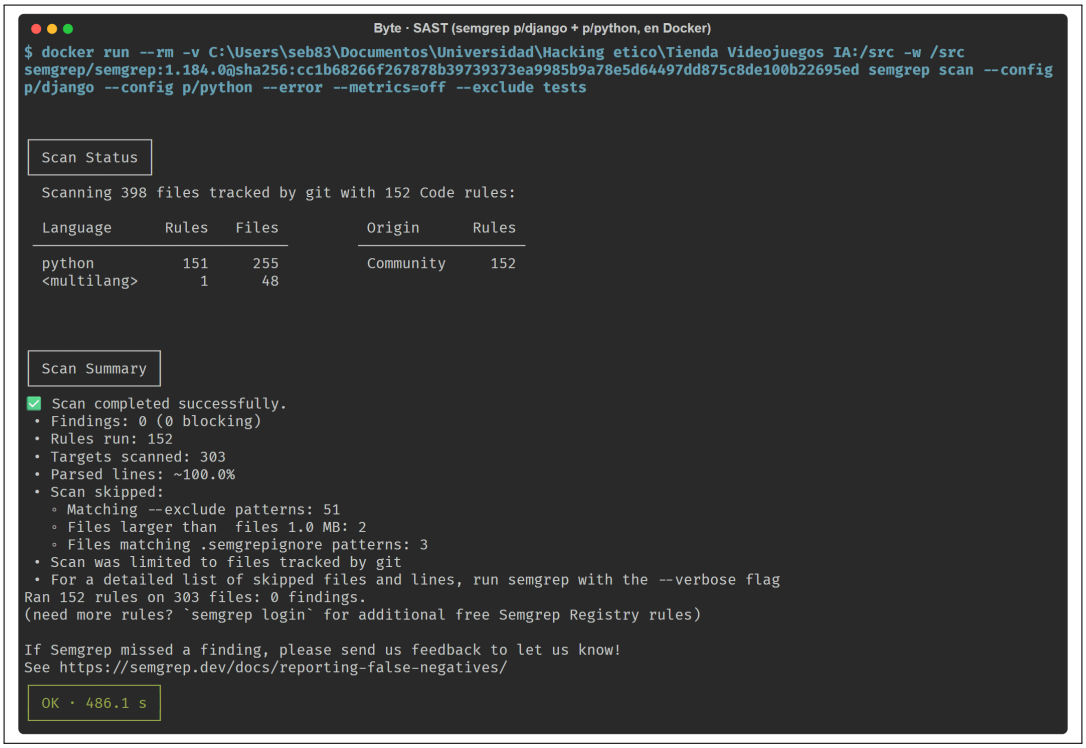


Nota. Captura de la Fase 2 tomada con la sesión real del usuario demo del rol vendedor (`docs/capturas/fase-02/403-vendedor-en-auditoria-escritorio-oscuro.jpg`).

Un error de la IA, y por qué importa. El commit de evidencias de esta fase afirmó «semgrep sin hallazgos», pero esa ejecución había reportado tres: una posible redirección abierta en el *middleware* del personal y dos usos de SHA-1. Se detectó al revisar la salida guardada, se corrigió en el commit siguiente y la bitácora lo registra con una nota de transparencia. La Figura 7 muestra la ejecución posterior, ya sin hallazgos.

Figura 7

Ejecución de semgrep tras corregir los hallazgos de la Fase 2



Nota. Imagen de terminal de `docs/capturas/fase-02/10-semgrep.svg`: cero hallazgos en 152 reglas.

Lección para replicar

La IA puede resumir mal un resultado aunque la herramienta lo haya reportado bien. Guarde siempre la salida completa de cada verificación (no solo el resumen que escribe la IA) y contrástela antes de dar por cerrada una fase.

Fase 3: catálogo y precios

Fragmento 6 del prompt maestro. Catálogo y descubrimiento (secciones 4.1 y 4.2)

Precio según configuración: precio base + modificadores por atributo, o precio fijo por variante; el selector de la ficha de producto actualiza precio, disponibilidad, SKU e imagen en vivo (HTMX), y deshabilita combinaciones que no existen.

[…]

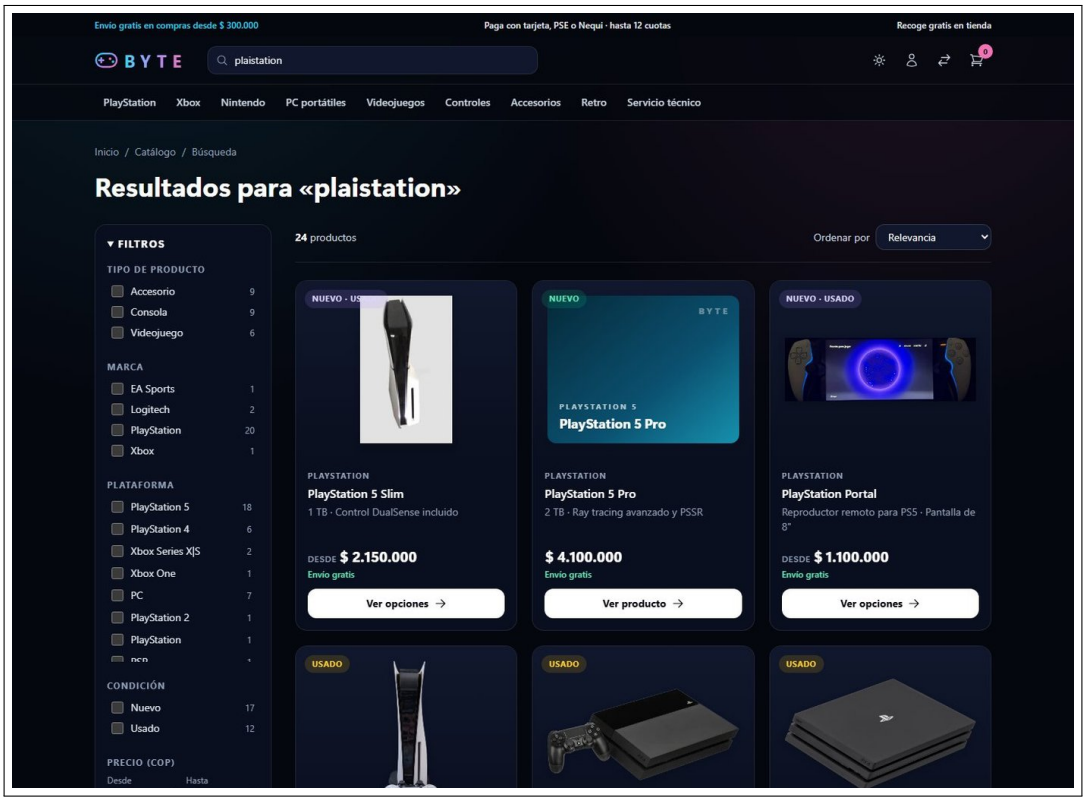
Búsqueda full-text con tolerancia a errores (pg_trgm) y autocompletado.

Cómo se implementó. La búsqueda usa las capacidades nativas de PostgreSQL (texto completo y similitud por trigramas) en lugar de un motor externo, para no agregar un servicio más que desplegar y asegurar (PostgreSQL Global Development Group, 2024). Un ajuste fino

salió de las propias pruebas: con un umbral de similitud de 0,45, la búsqueda «timon» encontraba «verde limón»; con 0,55 dejó de hacerlo y siguió encontrando «plaistation» (Figura 8). La ficha de producto actualiza precio, disponibilidad y SKU al cambiar la configuración y deshabilita las combinaciones que no existen (Figura 9). El historial de precios es de solo inserción y el «precio más bajo en 30 días» se calcula a partir de él, de modo que no se puede inflar un descuento, en línea con el Estatuto del Consumidor (Congreso de Colombia, 2011).

Figura 8

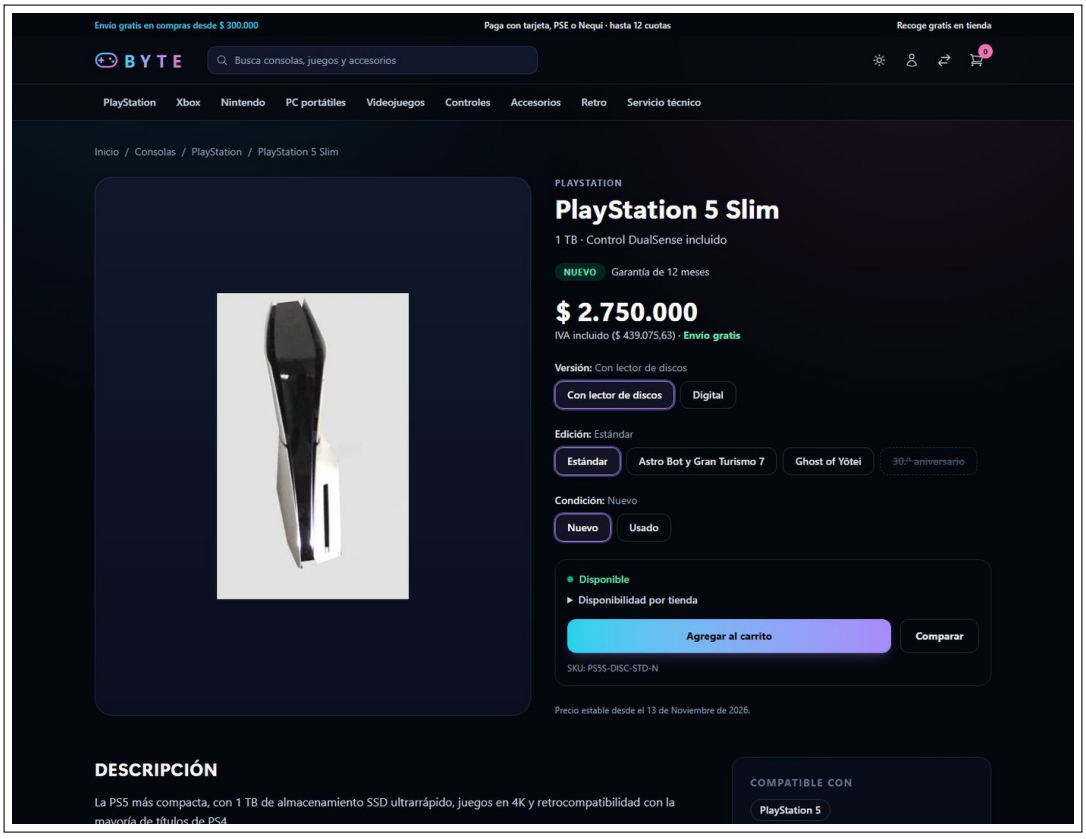
Búsqueda con error de digitación: «plaistation»



Nota. Captura de la Fase 3 (`docs/capturas/fase-03/busqueda-con-error-de-tipeo-escritorio-oscuro.jpg`).

Figura 9

Ficha de producto con configurador de variantes



Nota. Captura de la Fase 3 (`docs/capturas/fase-03/ficha-ps5-slim-escritorio-oscuro.jpg`), recortada.

Fase 4: carrito, checkout y pagos

Fragmento 7 del prompt maestro. Carrito y checkout (sección 4.3)

El servidor recalcula siempre precios, descuentos, impuestos y envío; jamás confía en valores del cliente.

[…]

Pagos con pasarela en sandbox; webhooks con verificación de firma e idempotencia; nunca almacenar datos de tarjeta (solo tokens/referencias de la pasarela).

Cómo se implementó. El pedido se crea en un único servicio transaccional que recalcula en el servidor precios, cupón, envío e IVA, y reserva el inventario «todo o nada» bloqueando las filas en orden para evitar interbloqueos, lo que cubre también el requisito de concurrencia de la sección 7 (A06). La pasarela Wompi (Wompi, s. f.) quedó detrás de la interfaz `PaymentGateway`

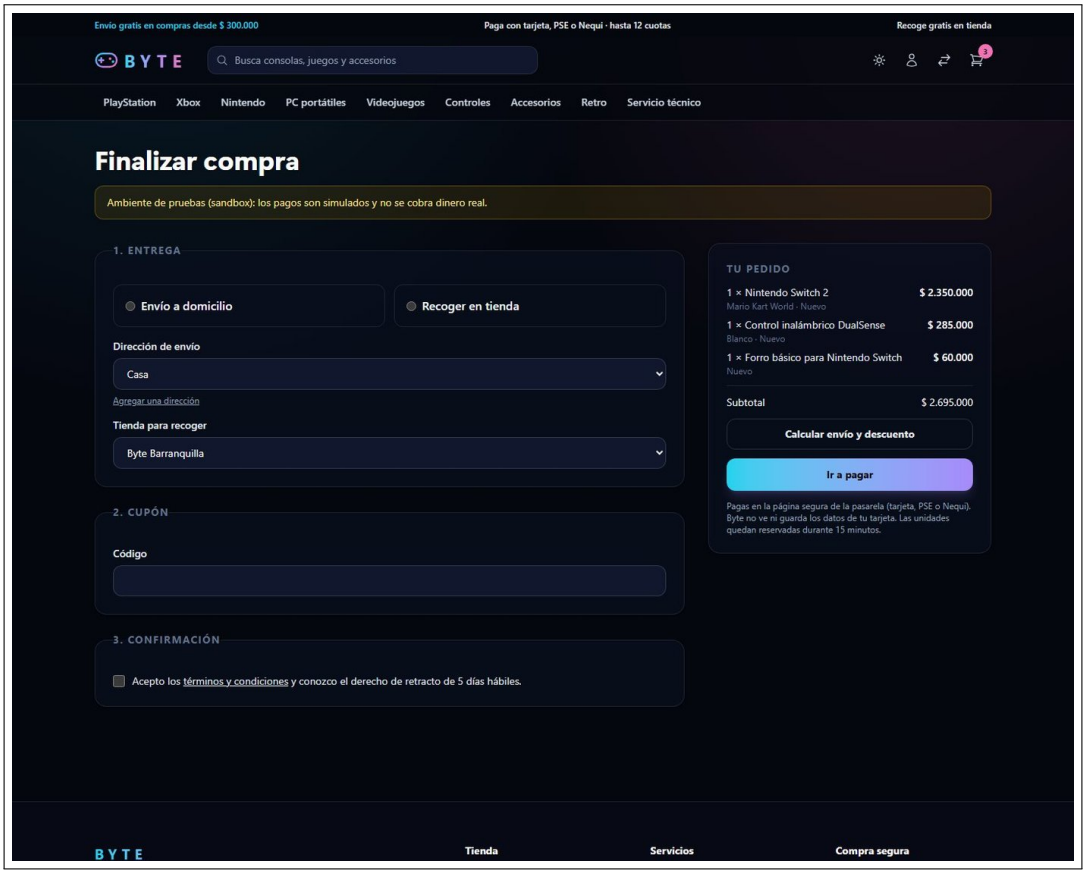
que pedía la sección 1, con un simulador para desarrollo que emite eventos firmados por el mismo camino que un webhook real. El enlace de pago lleva la firma de integridad que exige la pasarela:

```
firma = SHA-256( referencia + monto_en_centavos + "COP" + secreto_de_integridad )
```

Un pago solo se aplica con firma válida, referencia existente y monto exacto; si no, se rechaza y se registra un evento de seguridad. Las Figuras 10 y 11 muestran el checkout y la pasarela de pruebas.

Figura 10

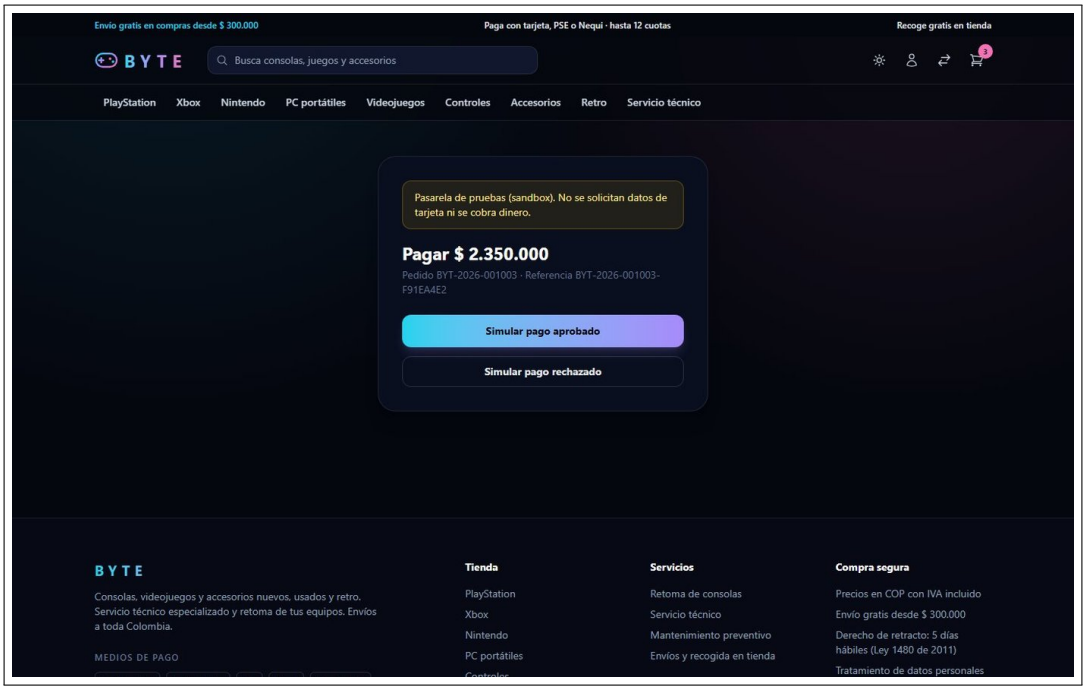
Checkout con envío, cupón y aviso de pagos de prueba



Nota. Captura de la Fase 4 (`docs/capturas/fase-04/checkout-escritorio-oscuro.jpg`), recortada.

Figura 11

Pasarela de pruebas del simulador de pagos



Nota. Captura de la Fase 4 (`docs/capturas/fase-04/pasarela-de-pruebas-escritorio-oscuro.jpg`).

Fase 5: panel administrativo

Fragmento 8 del prompt maestro. Panel de administración (sección 5)

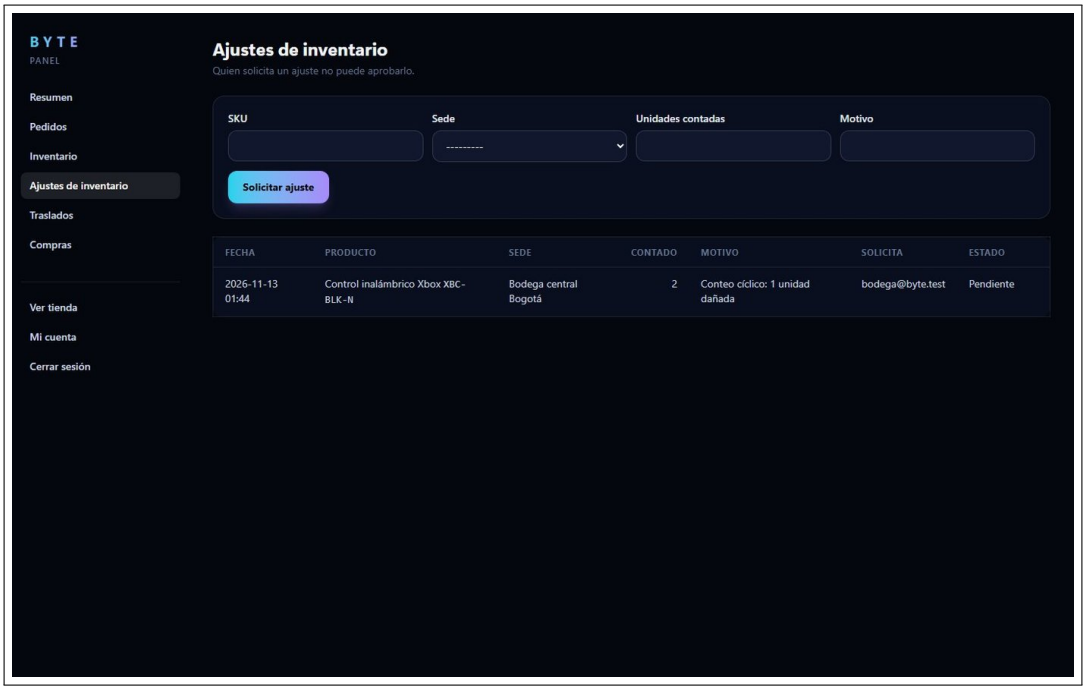
Inventario: multi-sede/bodega, stock disponible/reservado/en tránsito, kardex de movimientos (entrada, salida, ajuste, traslado, devolución) con usuario y motivo, […] ajustes con aprobación […].

POS (punto de venta): venta rápida en mostrador con búsqueda/lector de código de barras, cliente opcional, varios medios de pago, apertura y cierre de caja con arqueo.

Cómo se implementó. La IA aplicó la separación de funciones de la sección 3.2, con el umbral `[X]` fijado en 15 % y configurable por entorno: quien solicita un ajuste de inventario, una orden de compra o un cupón con descuento alto no puede aprobarlo, y la base de datos lo garantiza con restricciones `CHECK` aunque alguien salte el servicio (Figura 12). Cada vista del panel declara sus permisos y filtra por las sedes asignadas al usuario: un objeto de otra sede responde 404. El punto de venta reutiliza el mismo flujo de pedidos y reservas de la web, de modo que todas las salidas de inventario quedan en el mismo kardex (Figura 13).

Figura 12

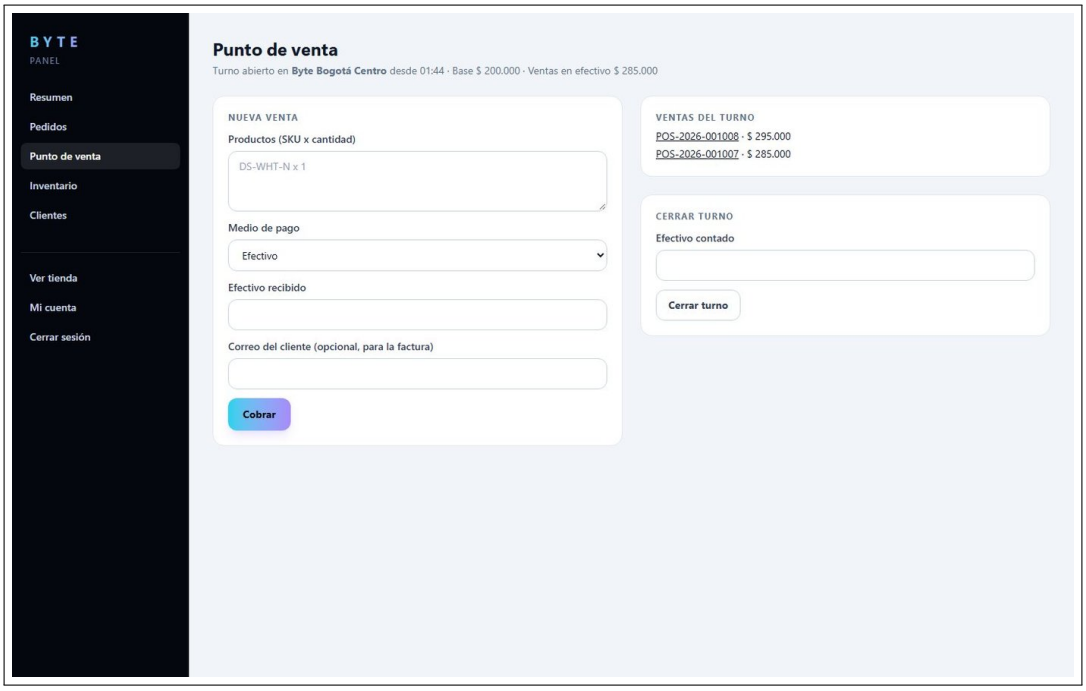
Un bodeguero solicita un ajuste de inventario pero no puede aprobarlo



Nota. Captura de la Fase 5 (`docs/capturas/fase-05/panel-ajustes-bodega-no-aprueba-escritorio-oscuro.jpg`).

Figura 13

Punto de venta con turno de caja abierto



Nota. Captura de la Fase 5 (`docs/capturas/fase-05/panel-pos-escritorio-claro.jpg`).

Fase 6: módulos de valor agregado

Fragmento 9 del prompt maestro. Cuenta de cliente (sección 4.4)

Retoma (trade-in): el cliente cotiza su consola/juego usado (modelo, estado, accesorios incluidos) → obtiene oferta estimada → agenda revisión → el técnico confirma → crédito en tienda o pago.

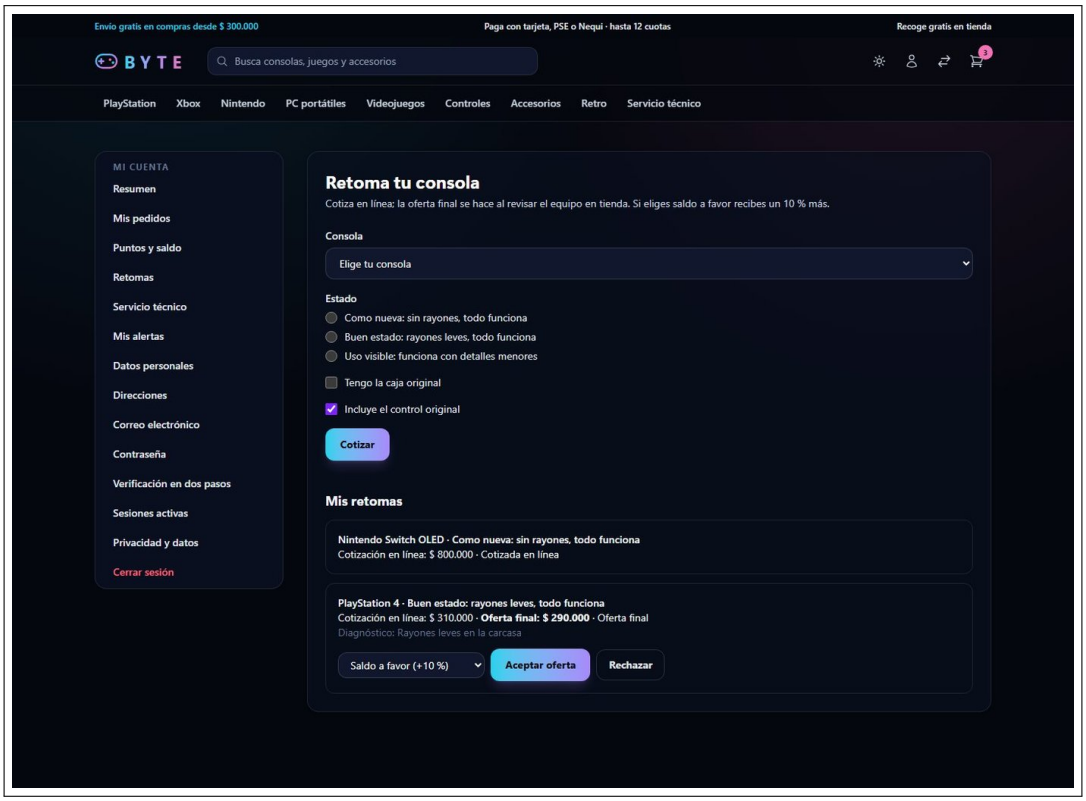
Servicio técnico: abrir ticket de reparación o garantía, adjuntar fotos (validadas), seguir estado.

Cómo se implementó. La retoma calcula una oferta estimada a partir del precio de la consola usada y del estado declarado, y la oferta final la confirma el técnico en tienda (Figura 14). El servicio técnico tiene órdenes con código de seguimiento, cotización que aprueba el cliente y garantía (Figura 15). Los puntos de fidelidad y el saldo a favor se modelaron como libros contables de solo inserción, en los que cada movimiento tiene origen, autor y fecha, y un bloqueo por usuario evita que dos operaciones simultáneas calculen mal el saldo. Las devoluciones respetan los plazos de retracto de la Ley 1480 y el reembolso necesita la aprobación de una persona distinta de quien recibió el producto, según los flujos de aprobación de la

sección 7 (A06).

Figura 14

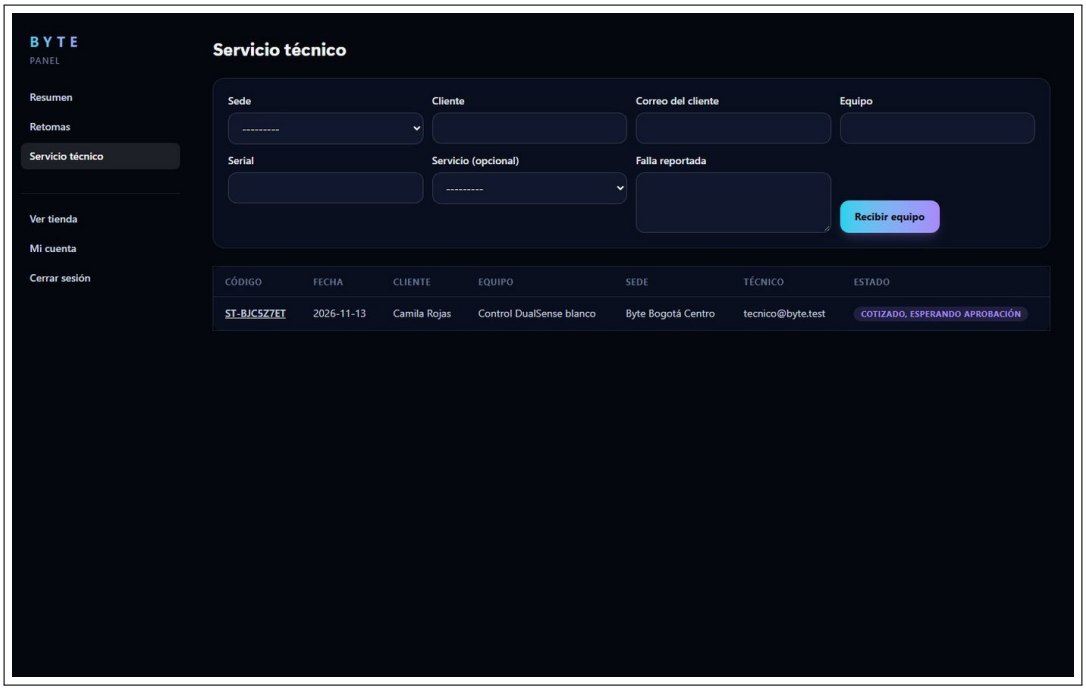
Cotización de una retoma desde la cuenta del cliente



Nota. Captura de la Fase 6 (`docs/capturas/fase-06/cuenta-retomas-escritorio-oscuro.jpg`), recortada.

Figura 15

Bandeja de servicio técnico en el panel



Nota. Captura de la Fase 6 (`docs/capturas/fase-06/panel-servicio-tecnico-escritorio-oscuro.jpg`).

Fase 7: big data e inteligencia de negocio

Fragmento 10 del prompt maestro. Pipeline de datos (sección 6.2)

Captura de eventos: modelo Event […] registrado de forma asíncrona y sin PII cruda (ID de usuario seudonimizado).

Bronze: extracción incremental desde PostgreSQL a archivos Parquet particionados por fecha.

Silver: limpieza, deduplicación y tipado con Polars.

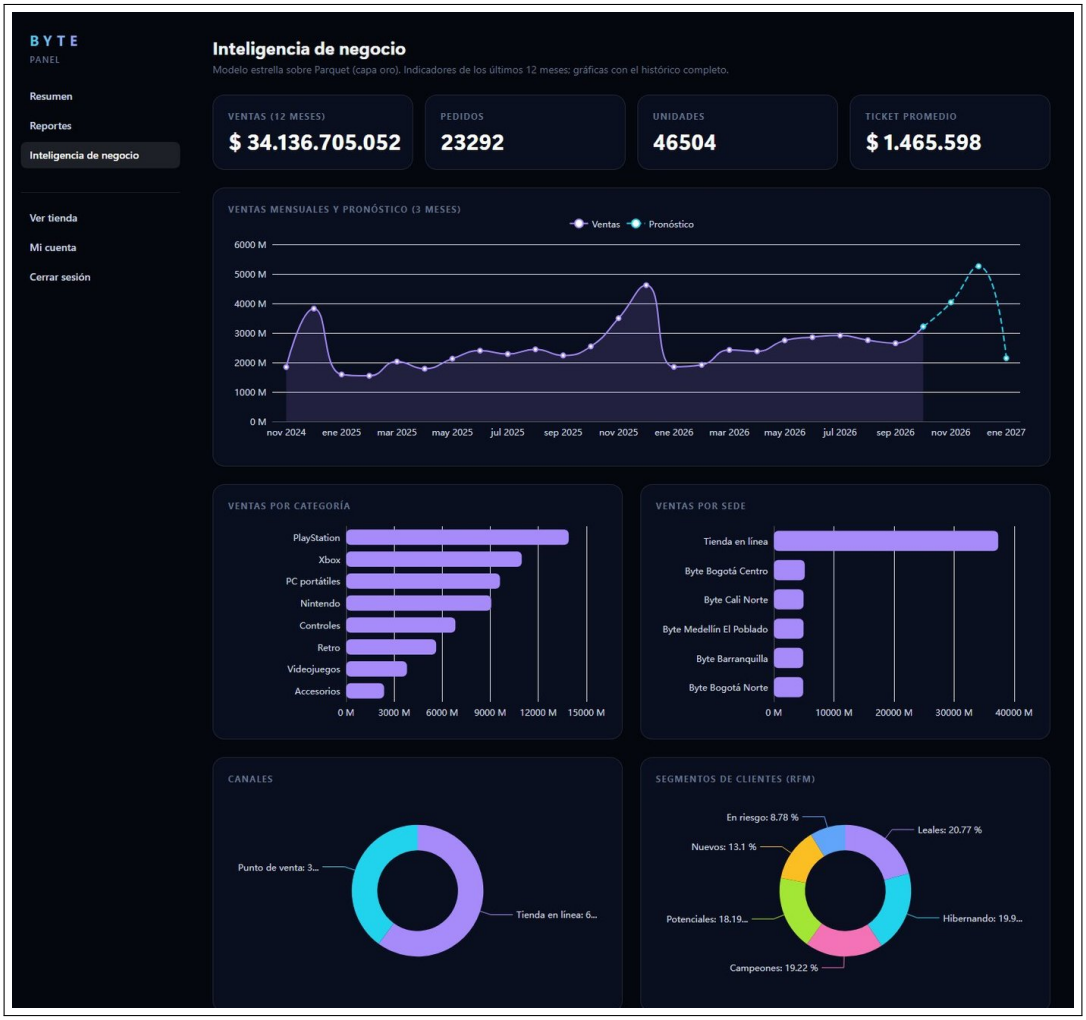
Gold: modelo estrella (fact_sales, fact_inventory_snapshot, fact_events, dim_product, dim_customer, dim_date, dim_store, dim_channel) consultado con DuckDB.

Cómo se implementó. La IA construyó el lago de datos con capas bronce, plata y oro sobre archivos Parquet, sin servidores nuevos, y seudonimizó a los clientes antes de extraerlos, de modo que el lago no contiene datos personales. Usó DuckDB también para la capa de plata, en lugar de Polars, y lo documentó en el ADR-0017: con el volumen del proyecto, una sola herramienta embebida bastaba. bandit señaló que el SQL de DuckDB se armaba con cadenas formateadas; en lugar de silenciar la alerta, la IA reescribió las consultas con SQL constante

sobre vistas registradas, como pedía la sección 7 (A05). El tablero (Figura 16) muestra ventas mensuales con pronóstico, categorías, sedes, canales y segmentos RFM.

Figura 16

Tablero de inteligencia de negocio



Nota. Captura de la Fase 7 (`docs/capturas/fase-07/panel-inteligencia-de-negocio-escritorio-oscuro.jpg`), recortada. Los datos son sintéticos.

Fase 8: endurecimiento de seguridad

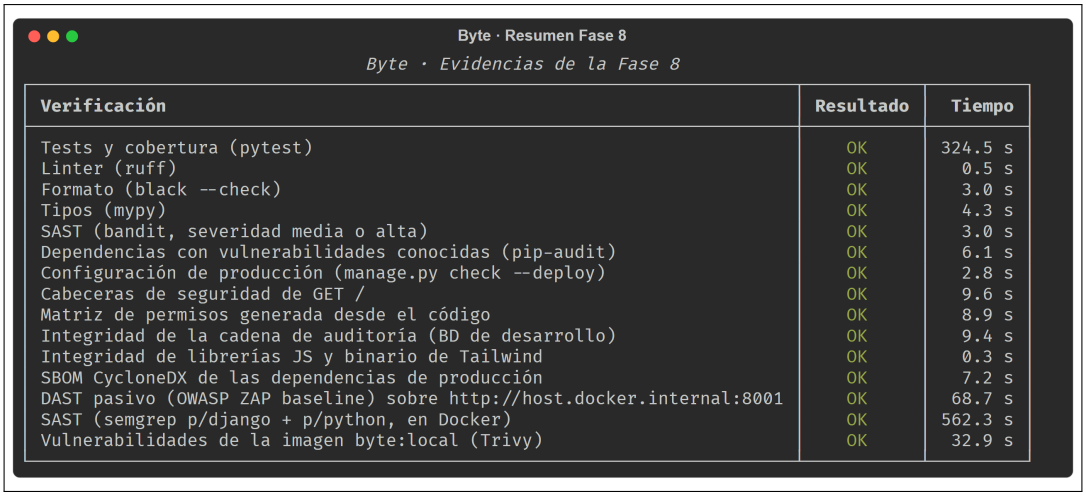
Fragmento 11 del prompt maestro. Seguridad (sección 7)

> Genera docs/seguridad.md con una matriz de trazabilidad: categoría OWASP → riesgo en esta app → control implementado → archivo/código → test que lo verifica. […]
>
> En CI: bandit, semgrep –config p/django, pip-audit, manage.py check –deploy, y OWASP ZAP baseline contra el entorno de staging o un contenedor levantado en el pipeline. El pipeline falla con hallazgos altos.

Cómo se implementó. La IA escribió la matriz control por control en `docs/seguridad/owasp-top10-2025.md`, el modelo de amenazas STRIDE con su riesgo residual y una prueba automática por categoría OWASP. Agregó al script de evidencias la generación del SBOM CycloneDX y un escaneo ZAP baseline (ZAP Development Team, s. f.) contra la aplicación levantada en local. El resultado fue cero fallos en ZAP (55 reglas superadas y 13 advertencias, todas analizadas en la bitácora), cero hallazgos de semgrep en 414 archivos y cero vulnerabilidades altas o críticas corregibles en Trivy (Figura 17).

Figura 17

Resumen de las quince verificaciones de la Fase 8



Nota. Imagen de terminal generada por `scripts/evidencia.py` (`docs/capturas/fase-08/00-resumen.svg`).

Conviene precisar el alcance de esta fase: son verificaciones defensivas y automáticas, hechas desde el código y con escaneo pasivo. No sustituyen una prueba de penetración, que intenta explotar la aplicación desde la perspectiva de un atacante; esa prueba se documenta en la

sección «Pruebas de penetración».

Fase 9: despliegue en Render

Fragmento 12 del prompt maestro. Despliegue y disponibilidad (sección 10)

Archivo de infraestructura como código (render.yaml Blueprint o equivalente) con: servicio web (Gunicorn), worker de Celery, Celery Beat (o cron de la plataforma), PostgreSQL gestionado y Redis/Key-Value gestionado.

[…]

Al final, despliega la aplicación, ejecuta seed_demo (y un seed_bigdata reducido acorde al plan) en producción, verifica manualmente el flujo completo (registro → compra → panel → analítica) y entrégame la URL pública y el usuario demo de cada rol.

Cómo se implementó. La IA definió los cinco servicios en un Blueprint (Render, s. f.): base de datos, Valkey, servicio web, worker y programador de tareas, con la base de datos y Valkey sin acceso público. Para cumplir la regla de la sección 0 sobre secretos, todos los secretos los genera la propia plataforma (`generateValue`), de modo que nadie los escribió ni quedaron en el repositorio:

```
      - key: DJANGO_SECRET_KEY
        generateValue: true
      - key: BYTE_FIELD_ENCRYPTION_KEYS
        generateValue: true # se deriva una clave Fernet (apps/core/crypto.py)
      - key: BYTE_PAYMENT_GATEWAY
        value: fake # cambiar a wompi al cargar las llaves del sandbox
      - key: BYTE_DEMO_PASSWORD
        generateValue: true # visible en el panel de Render (usuarios demo)
```

El primer despliegue falló, y los dos siguientes también. La IA leyó los registros de Render, encontró la causa de cada fallo y lo corrigió con un commit (Tabla 7).

Tabla 7

Fallos del despliegue en Render y su corrección

Síntoma	Causa	Corrección
El pre-despliegue salió con estado 2.	Render ejecuta el comando sin *shell*: el `&&` llegó a `manage.py migrate` como argumento.	Envolver el comando en `sh -c`.
Salió con estado 127.	Render tampoco interpreta comillas: `sh` recibió la cadena con las comillas literales.	Mover los comandos a scripts en la carpeta `deploy/`.
Error de gunicorn al arrancar (no fatal).	Su *socket* de control intentaba escribir en un directorio de solo lectura.	Desactivar el *socket* de control, que no se usa.

Nota. Commits `645ae51`, `a7f82a2` y `31e5cc6` del repositorio.

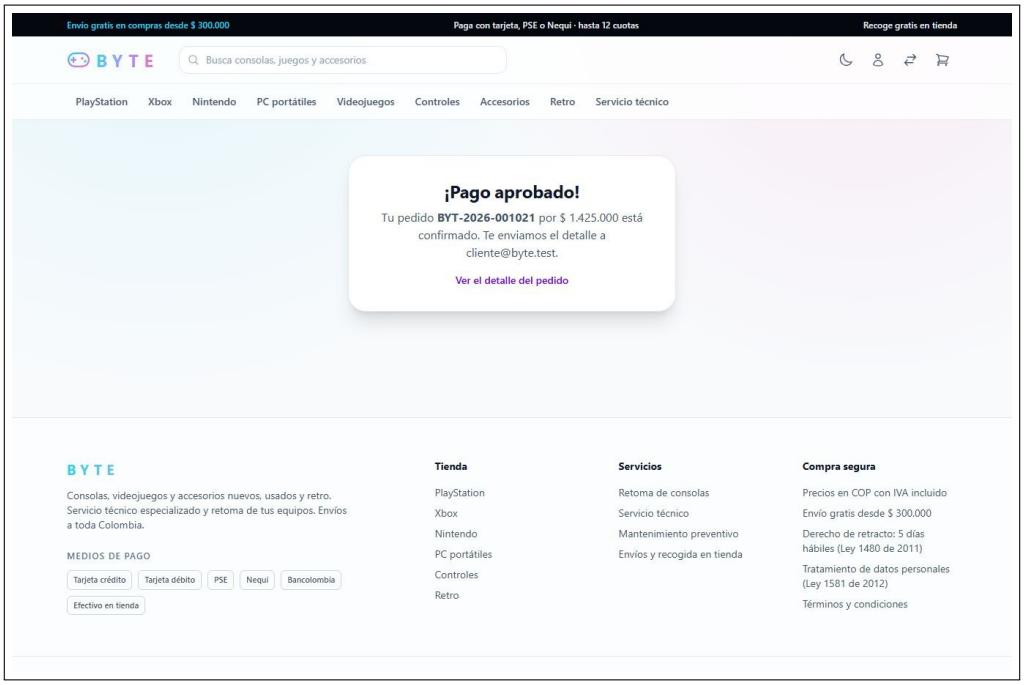
El script final de pre-despliegue quedó así:

```
#!/bin/sh
# Render pre-deploy: migrations, roles and demo data (idempotent).
set -eu
python manage.py migrate --noinput
python manage.py bootstrap_demo --sin-bi
```

Antes del despliegue, la prueba de extremo a extremo con Playwright recorrió la compra completa en siete pasos sin errores: inicio, búsqueda, ficha, carrito de invitado, inicio de sesión con fusión del carrito, checkout, pasarela de pruebas, pago aprobado, pedido y factura (Figuras 18 y 19). Ya en Render, la aplicación quedó disponible en https://byte-web-v9h4.onrender.com (Figura 20), con las cabeceras de seguridad activas y los servicios de base de datos, Valkey, worker y programador en funcionamiento.

Figura 18

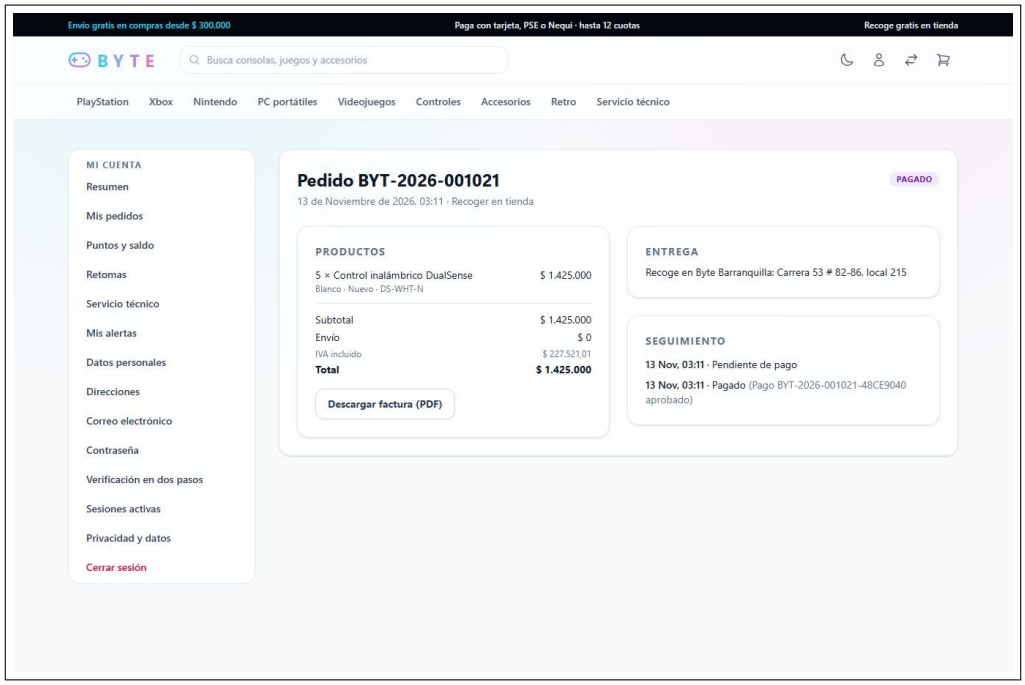
Pago aprobado durante la prueba de extremo a extremo



Nota. Captura tomada por `scripts/e2e.py` (`docs/capturas/fase-09/e2e-06-pago-aprobado.jpg`).

Figura 19

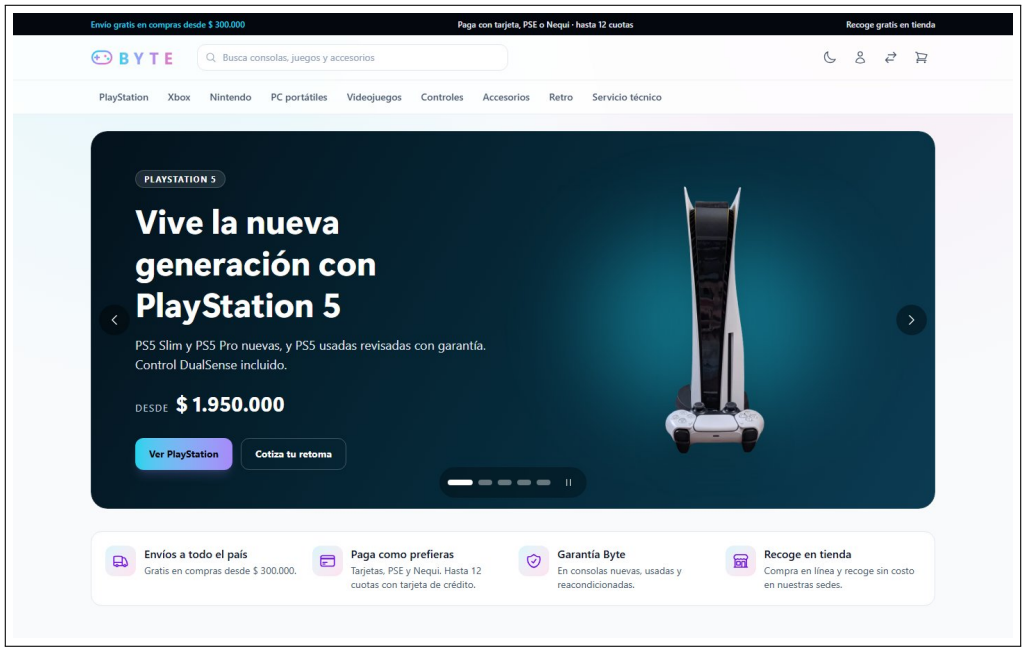
Detalle del pedido pagado en la cuenta del cliente



Nota. Captura tomada por `scripts/e2e.py` (`docs/capturas/fase-09/e2e-07-pedido.jpg`).

Figura 20

La aplicación desplegada en Render



Nota. Captura de https://byte-web-v9h4.onrender.com tomada con Playwright después del despliegue.

Fase 10: documentación final

Fragmento 13 del prompt maestro. Entregable de la Fase 10 (sección 11)

README, manual de usuario por rol (con capturas), manual técnico, docs/seguridad.md, docs/big-data.md, guion de sustentación

La última fase produjo el README, los manuales del cliente y del personal por rol, el manual técnico y el guion de sustentación, todos en `docs/`. Como la regla de la sección 0 obligaba a documentar al cerrar cada fase, esta fase consistió sobre todo en consolidar, no en reconstruir.

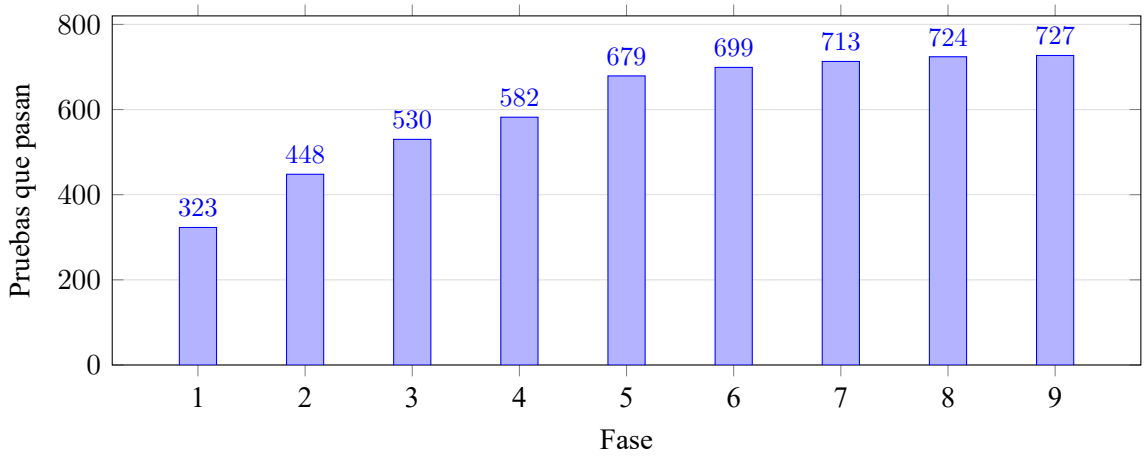
Resultados

Pruebas y verificaciones

La Figura 21 muestra cómo creció la suite de pruebas fase a fase. La cobertura se mantuvo por encima del 93 % desde la Fase 2, muy por encima de la meta del 80 % que fijó el prompt maestro.

Figura 21

Número de pruebas automáticas al cerrar cada fase



Nota. Elaboración propia con los datos de la Tabla 5. La Fase 10 no agregó pruebas.

La Tabla 8 consolida las cifras finales del repositorio y de las verificaciones.

Tabla 8

Cifras finales del proyecto

Indicador	Valor
Pruebas automáticas y cobertura	727 pruebas; 93 % de cobertura
Código Python (apps, configuración, scripts y pruebas)	388 archivos; unas 25 800 líneas
Apps de Django, plantillas y migraciones	21 apps; 93 plantillas; 40 migraciones
Decisiones de arquitectura	17 ADR
Historia del repositorio	28 commits en 10 ramas de fase
semgrep	0 hallazgos en 152 reglas y 414 archivos
OWASP ZAP baseline	0 fallos; 55 reglas superadas; 13 advertencias analizadas
Trivy y pip-audit	0 vulnerabilidades altas o críticas corregibles; 0 dependencias vulnerables
Prueba de extremo a extremo	7 pasos sin errores
Despliegue	https://byte-web-v9h4.onrender.com

Nota. Conteos obtenidos con `git ls-files` y `git log` sobre la rama `main`; resultados de seguridad tomados de la bitácora de la Fase 8.

Cómo resolvió la IA las opciones abiertas del prompt

Donde el prompt maestro ofrecía alternativas, la IA aplicó la regla «elige la más simple y segura, y documéntala como ADR». La Tabla 9 resume esas decisiones.

Tabla 9

Opciones abiertas del prompt maestro y decisión tomada

Opción en el prompt	Decisión	Motivo (ADR)
Django 5.2 LTS «o la LTS vigente»	Django 5.2.19 LTS	Soporte durante todo el proyecto (ADR-0002).
CSP nativa de Django o django-csp	django-csp	Django 5.2 no trae CSP nativa (ADR-0002).
django-guardian o servicio propio	Servicio propio	El alcance es por dueño o por sede; los *querysets* filtrados deniegan por defecto sin tablas de permisos por fila (ADR-0003).
uv o pip-tools	uv	*Hashes* y cuarentena de 7 días contra paquetes recién publicados (ADR-0005).
django-tailwind o CLI standalone	CLI standalone	Sin Node; menos superficie de ataque (ADR-0006).
Wompi o Mercado Pago	Wompi	Más directo para Colombia; queda detrás de una interfaz intercambiable (ADR-0007).
Redis	Valkey 8	Compatible con Redis y es el servicio gestionado de Render (ADR-0009).
Turnstile o hCaptcha	Turnstile	No rastrea al usuario y casi nunca le pide resolver un reto (ADR-0012).
Polars para la capa de plata	DuckDB	Una sola herramienta embebida basta para el volumen del proyecto (ADR-0017).
Descuento > [X] % con aprobación	15 %, configurable	Variable `BYTE_HIGH_DISCOUNT_APPROVAL_PCT`.
`[PLATAFORMA_DE_DESPLIEGUE]`	Render	Plataforma por defecto del propio prompt.

Nota. Elaboración propia a partir de `docs/adr/`.

Criterios de aceptación

El prompt maestro terminaba con nueve criterios de aceptación. La Tabla 10 los contrasta con lo logrado, sin maquillar los que quedaron parciales.

Tabla 10

Criterios de aceptación del prompt maestro

Criterio	Estado	Observación
Desplegada y accesible por HTTPS con URL pública	Cumplido	En Render, con HSTS y TLS 1.2/1.3.
Un usuario demo por rol, cada uno con su alcance	Cumplido	Ocho usuarios; la contraseña la genera Render.
Configurar una consola, ver el precio cambiar, comprar y seguir el pedido	Cumplido con matiz	Se paga con el simulador; Wompi requiere cargar las llaves del sandbox.
Inventario sin sobreventa bajo concurrencia, con prueba	Cumplido	Reserva todo o nada con bloqueo de filas.
Panel con catálogo, inventario, ventas, compras, POS, clientes, retomas y reportes	Cumplido	Con alcance por sede y separación de funciones.
Analítica con al menos 500 000 pedidos	Parcial	El histórico sintético tiene unas 70 000 líneas de venta en dos años.
`check --deploy`, bandit, pip-audit y ZAP sin hallazgos altos	Cumplido	Ver Figura 17.
Matriz OWASP 2025 con controles y pruebas	Cumplido	`docs/seguridad/owasp-top10-2025.md`.
Cobertura de al menos 80 % y CI en verde	Cumplido con matiz	Cobertura del 93 %; el pipeline está configurado y su ejecución en GitHub queda por confirmar.

Nota. Criterios tomados de la sección 12 del prompt maestro (Apéndice A).

Lecciones aprendidas

De la experiencia se desprenden ocho recomendaciones para quien quiera construir una aplicación seria con *vibe coding*:

1. **Invierta en el prompt maestro.** Una especificación larga y concreta reemplazó decenas de conversaciones. Los requisitos verificables («tests que prueban 403/404 cambiando UUID») rinden más que los genéricos («que sea seguro»).

2. **Trabaje por fases con una puerta humana.** La obligación de detenerse al final de cada fase permitió revisar y corregir el rumbo antes de que los errores se acumularan.

3. **Verifique, no confíe.** La IA resumió mal un resultado de semgrep en la Fase 2; la salida completa guardada lo delató. Las pruebas de extremo a extremo y las capturas también encontraron errores que el código no mostraba.

4. **Ponga la seguridad en el prompt, no al final.** Las cabeceras, la CSP, Argon2, la auditoría y la denegación por defecto se configuraron desde la Fase 1 porque el prompt las exigía.

5. **Haga persistentes las reglas.** `CLAUDE.md`, los ADR y la bitácora sobrevivieron a reinicios y a resúmenes de contexto; la conversación no.

6. **Automatice la evidencia.** Un script que corre todas las verificaciones y toma las capturas convierte el cierre de una fase en un solo comando y produce el material del informe.

7. **Deje opciones abiertas, pero exija que se justifiquen.** Los corchetes y las alternativas del prompt dieron margen a la IA, y la regla de los ADR dejó cada decisión explicada.

8. **Nunca entregue secretos por el chat.** Los secretos de producción los generó la plataforma y las llaves reales las carga la persona en el panel de Render.

Este trabajo también tiene límites. La IA tomó la mayoría de las decisiones técnicas dentro del marco fijado; una persona sin conocimientos de desarrollo y de seguridad habría tenido dificultades para evaluar sus propuestas o detectar sus errores. Algunos criterios quedaron parciales, como el volumen de datos de la analítica. Y, como muestra la sección siguiente, aprobar todas las verificaciones automáticas no garantizó que el sistema desplegado estuviera libre de fallas.

Pruebas de penetración

Con la aplicación construida y desplegada, la segunda parte del proyecto la sometió a una prueba de penetración. El propósito fue contrastar los controles descritos en las secciones anteriores con los intentos de un atacante, en el sistema real expuesto a internet y no en el entorno de desarrollo. Esta sección es el informe de esa prueba: alcance, metodología, resultados por fase, hallazgos con su evidencia y recomendaciones.

Resumen ejecutivo

La prueba se ejecutó el 13 de noviembre de 2026, entre las 11:13 y las 12:25 UTC, desde Kali Linux (OffSec, s. f.), contra https://byte-web-v9h4.onrender.com. Combinó pruebas de caja negra, pruebas autenticadas con los usuarios de demostración y revisión dirigida del código fuente (caja gris).

El control de acceso resistió todos los intentos: no se logró acceder a datos de otro cliente (IDOR) ni escalar de cliente a personal, y el personal no puede entrar sin el segundo factor. Las sondas de inyección SQL y de XSS no prosperaron. Los hallazgos se concentran en la autenticación (Tabla 11): la protección contra fuerza bruta del inicio de sesión no funciona en la instancia desplegada (PT-01) y las cuentas demo del personal usan un secreto TOTP publicado en el repositorio (PT-04), ambos de severidad media. Se registraron además dos observaciones informativas. Ningún hallazgo permitió comprometer la aplicación durante la prueba. PT-01 y PT-04 se corrigieron y PT-01 se re-probó con éxito contra producción (sección «Remediación y re-prueba»).

Tabla 11

Resumen de hallazgos de la prueba de penetración

ID	Hallazgo	OWASP 2025 (2021)	Severidad	Estado
PT-01	Protección contra fuerza bruta inefectiva en `/cuenta/login/`	A07 (A07)	Media (CVSS 5,3)	Corregido
PT-04	Secreto TOTP público en las cuentas demo del personal	A07 (A07)	Media (CVSS 4,8)	Corregido
PT-02	Panel del personal descubrible en `/panel/`	A02 (A05)	Informativa	Riesgo aceptable
PT-03	Filtro `|safe` sobre `help_text` en dos plantillas	A05 (A03)	Informativa	Abierto

Nota. Elaboración propia. El estado es posterior a la remediación. Las puntuaciones CVSS 3.1 son estimaciones de este informe (FIRST, 2019); la herramienta de la prueba no las calcula. PT-04 se identificó al revisar el código durante la redacción del informe. Se conserva la numeración de la prueba, por eso el orden de la tabla es por severidad.

Alcance y reglas de enfrentamiento

La prueba se limitó al host `byte-web-v9h4.onrender.com`, la instancia de producción desplegada en la Fase 9, propiedad del autor, que la autorizó para este ejercicio académico. Las reglas fueron:

1. Solo el host anterior; quedan fuera la infraestructura compartida de Render y Cloudflare, otros clientes de esas plataformas y cualquier servicio de terceros (Wompi, Cloudflare Turnstile).
2. Sin denegación de servicio: los escaneos se limitaron en concurrencia y las pruebas de fuerza bruta, a 30 intentos.
3. Pruebas autenticadas solo con los usuarios de demostración sembrados por `bootstrap_demo`; ningún dato real de clientes.
4. Revisión del código fuente permitida, para explicar la causa raíz de lo observado.

Metodología y herramientas

La prueba siguió las fases clásicas de un pentest y clasificó los resultados según el OWASP Top 10. El evaluador usó la edición 2021 (OWASP Foundation, 2021); como el prompt maestro exige la de 2025 (OWASP Foundation, 2025), este informe presenta ambas. Las herramientas fueron las de la Tabla 12. Cada artefacto quedó guardado con su *hash* SHA-256: 45 evidencias entre peticiones y respuestas HTTP, salidas de escáneres, análisis de código y capturas.

Tabla 12

Herramientas usadas en la prueba de penetración

Herramienta	Uso
Kali Linux	Sistema base del evaluador.
whois, dig y openssl	Reconocimiento pasivo: dominio, DNS, cadena CNAME y certificado TLS.
numasec	Marco de la operación: rastreo web, análisis de JavaScript, búsqueda de directorios, sondas de seguridad de aplicaciones y registro de evidencias por *hash*.
curl y scripts propios	Pruebas manuales: CSRF, inyección, límite de intentos e inicio de sesión autenticado.
Navegador	Capturas de pantalla de las páginas probadas.
pip-audit	Análisis de dependencias contra vulnerabilidades conocidas.

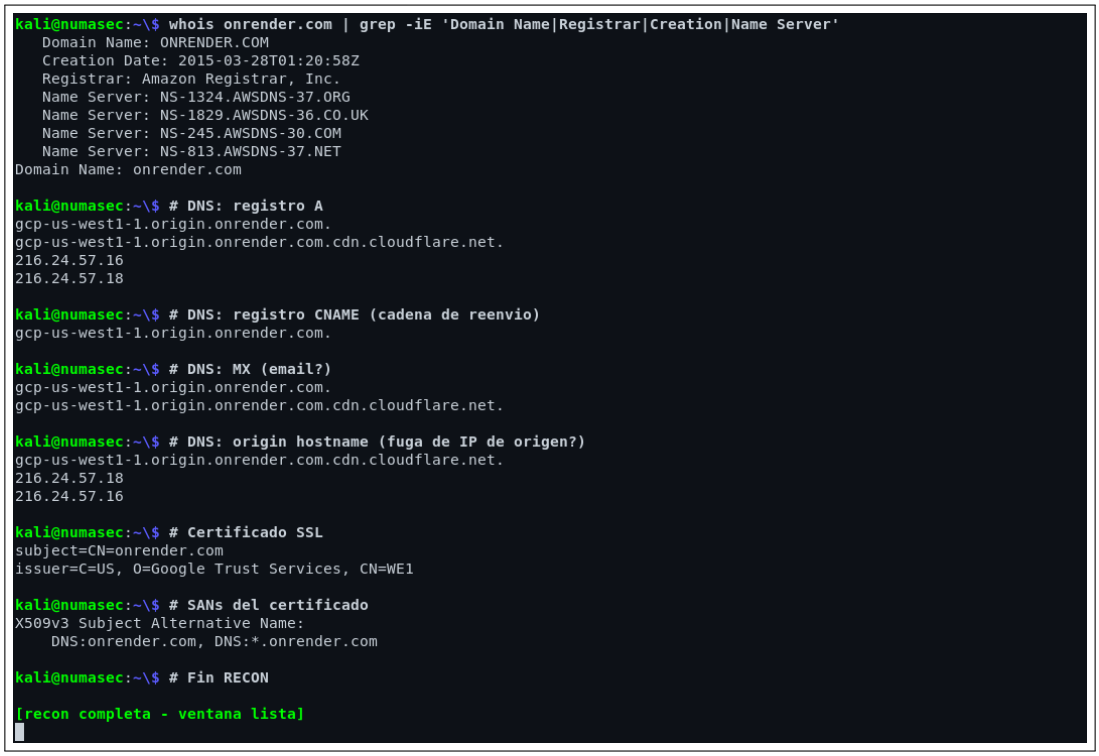
Nota. Elaboración propia a partir del paquete de evidencias de la prueba (`pentest-byte-web-v9h4-owasp.zip`).

Fase 1: reconocimiento

El reconocimiento pasivo (Figura 22) mostró que la aplicación corre en Render (`gcp-us-west1`) detrás de la red de Cloudflare: el nombre resuelve a 216.24.57.16 y 216.24.57.18, que son direcciones del borde de Cloudflare. El nombre de origen `gcp-us-west1-1.origin.onrender.com` también resuelve a Cloudflare, de modo que no hay fuga de la IP de origen que permita saltarse el cortafuegos de aplicaciones web (WAF). El certificado es un comodín `*.onrender.com` emitido por Google Trust Services, válido hasta el 20 de diciembre de 2026. El subdominio no tiene registros MX ni TXT.

Figura 22

Reconocimiento pasivo desde Kali: WHOIS, DNS y certificado



Nota. Captura de la terminal del evaluador (evidencia `e50beaef…`). Resultados detallados en la evidencia `bddbd12d….`

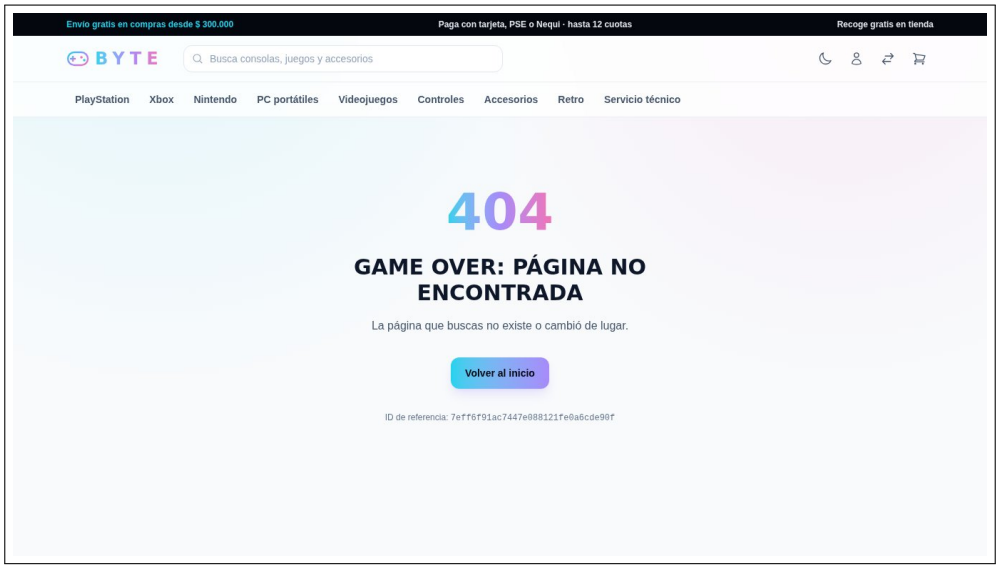
Fase 2: escaneo y enumeración

El rastreo web encontró 200 URL y el análisis de la superficie identificó 215 rutas y 81 formularios. Las entradas más relevantes fueron la búsqueda (`/buscar/?q=`), las sugerencias, los filtros del catálogo, el parámetro `sku` de la ficha, el carrito y los formularios de cuenta. El análisis de los cinco archivos JavaScript encontró 39 rutas y ninguna cadena con forma de secreto.

La búsqueda de directorios (169 peticiones) solo descubrió `/panel/`, que redirige al inicio de sesión (PT-02). El administrador de Django no está en `/admin/`, que responde con la página 404 propia y sin trazas de error (Figura 23), tal como pedía el prompt maestro. No hay `robots.txt` ni `sitemap.xml`.

Figura 23

La ruta `/admin/` responde con la página 404 propia



Nota. Captura del evaluador (evidencia `2a175426…`), recortada. El panel de Django vive en una ruta oculta configurada por variable de entorno.

Fase 3: análisis de vulnerabilidades

Las sondas automáticas probaron inyección SQL, XSS, autenticación débil, IDOR y CORS. La de IDOR no se ejecutó porque ninguna ruta usa identificadores numéricos, y la de CORS no encontró cabeceras permisivas. Las otras tres generaron candidatos, que se verificaron a mano y se descartaron (Tabla 13). La Figura 24 resume las comprobaciones manuales y la Figura 25, la búsqueda con un *script*, que la aplicación muestra como texto.

Tabla 13

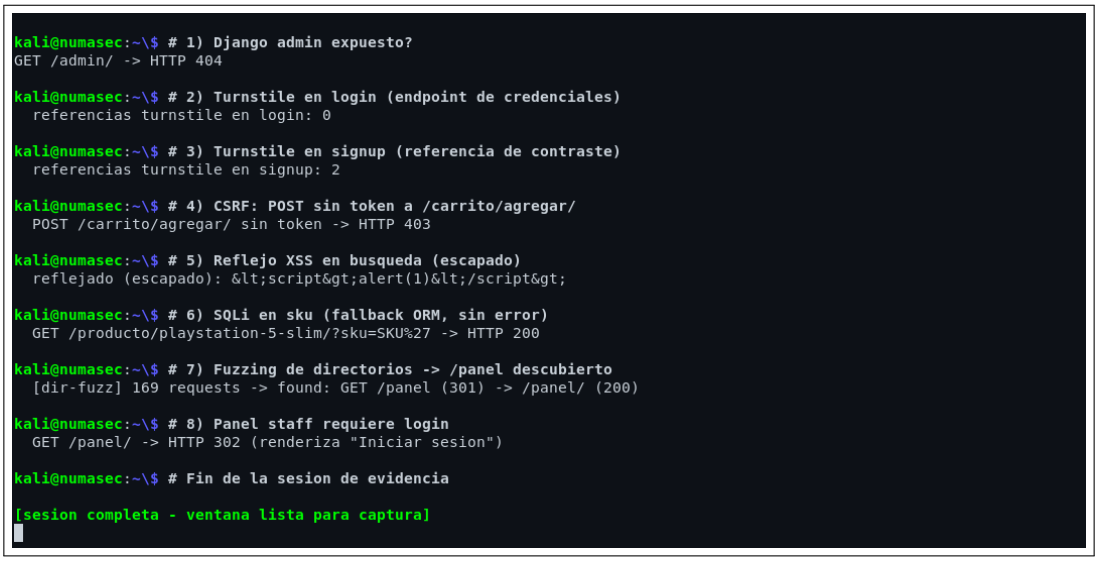
Candidatos de las sondas automáticas y verificación manual

Candidato	Señal	Verificación
Inyección SQL en `/buscar/?q=' OR 1=1--`	HTTP 403	Bloqueado por el WAF de Cloudflare, sin mensaje de error de base de datos. Con `?sku=SKU'` y `?orden=` arbitrario la aplicación responde 200 con su valor por defecto: el ORM no concatena la entrada. Descartado.
XSS reflejado en `/buscar/?q=<script>`	El marcador aparece en la respuesta	Aparece codificado (`<script>`) por el autoescapado de Django. Descartado.
Autenticación débil en `/cuenta/login/`	HTTP 403 y señal de *token*	El 403 es la protección CSRF; la aplicación no usa JWT. Descartado.

Nota. Elaboración propia a partir de la evidencia `12409d81…` (sondas) y `e0a6d465…` (controles positivos).

Figura 24

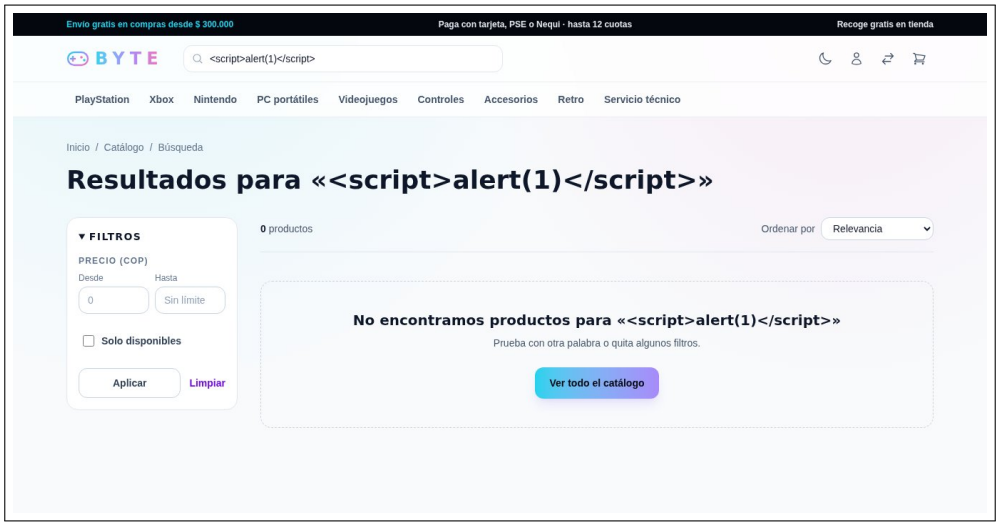
Comprobaciones manuales de OWASP desde la terminal de Kali



Nota. Captura de la terminal del evaluador (evidencia `3c128e69…`), recortada.

Figura 25

La búsqueda con `<script>alert(1)</script>` se muestra como texto



Nota. Captura del evaluador (evidencia `4c23d747…`), recortada.

En la misma fase, `pip-audit` sobre `requirements/prod.txt` no encontró vulnerabilidades conocidas en las dependencias, y las librerías JavaScript se sirven localmente con su integridad verificada.

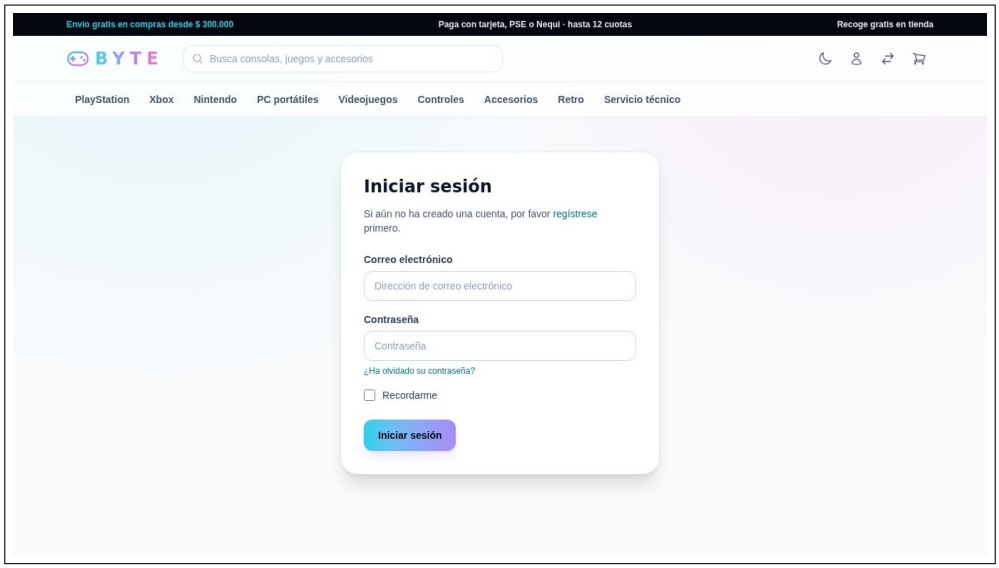
Fase 4: explotación

Fuerza bruta contra el inicio de sesión

La página de inicio de sesión no muestra Turnstile en el primer intento (Figura 26), mientras que el registro y el restablecimiento de contraseña sí lo hacen. El evaluador envió 30 intentos de inicio de sesión fallidos y rápidos, rotando el usuario, y luego 8 con el mismo usuario. Todos respondieron HTTP 200 con el mensaje de error genérico: ninguno recibió el 429 de bloqueo, aunque la configuración bloquea a los 5 fallos (Figura 27). El CAPTCHA adaptativo apareció de forma intermitente. Estos resultados sustentan el hallazgo PT-01.

Figura 26

Inicio de sesión sin CAPTCHA en el primer intento



Nota. Captura del evaluador (evidencia `fa8bbeb4…`), recortada.

Figura 27

Intentos de inicio de sesión fallidos sin bloqueo



Nota. Captura de la terminal del evaluador (evidencia `3ca17c0e…`). La ventana muestra los primeros 26 de los 30 intentos; el registro completo está en la evidencia `bee2e208…`.

Pruebas autenticadas de control de acceso

Con los usuarios de demostración, el evaluador comprobó lo siguiente:

1. El cliente inicia sesión sin segundo factor, como corresponde a su rol.
2. Un usuario del personal (vendedor) no entra solo con la contraseña: la aplicación lo lleva a `/cuenta/2fa/authenticate/` y exige el código TOTP.
3. Escalamiento vertical: el cliente que pide `/panel/` recibe 403.
4. IDOR: la dirección propia del cliente responde 200; un UUID ajeno o el UUID nulo responden 404, sin revelar si el objeto existe. El código confirma que el *selector* filtra siempre por el usuario.
5. La API `/api/v1/` solo expone el esquema, reservado al personal; no hay datos expuestos.
6. Un POST a `/carrito/agregar/` sin *token* CSRF recibe 403.

Fase 5: post-explotación

Ningún intento dio un acceso mayor que el de los usuarios de demostración, de modo que no hubo persistencia ni movimiento lateral que evaluar. La fase se centró en el impacto. PT-01 permitiría automatizar el robo de credenciales por relleno (*credential stuffing*) contra las cuentas de clientes, pues no hay CAPTCHA efectivo ni bloqueo. Para el personal el riesgo es menor, porque exige el segundo factor, salvo en las cuentas demo, cuyo factor es público (PT-04).

No se verificó, desde el lado del defensor, que la auditoría y las alertas registraran los intentos: desde fuera el registro no es observable (A09). Queda como verificación pendiente en la bitácora de eventos de seguridad del panel.

Hallazgos

PT-01. Protección contra fuerza bruta inefectiva en el inicio de sesión

Severidad: media (CVSS 3.1 estimado 5,3; AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N). **OWASP:** A07:2025 Fallas de autenticación.

Descripción. El bloqueo progresivo de django-axes y el CAPTCHA adaptativo no protegen el inicio de sesión en la instancia desplegada. Las Figuras 26 y 27 muestran la evidencia.

Causa raíz. Se confirmó en el código y en la configuración de despliegue:

```
AXES_LOCKOUT_PARAMETERS = [["username", "ip_address"]]  # user + IP combination
AXES_IPWARE_PROXY_COUNT = env.int("AXES_IPWARE_PROXY_COUNT", default=0)  # 1 behind
↪  Render's proxy
AXES_IPWARE_META_PRECEDENCE_ORDER = ("HTTP_X_FORWARDED_FOR", "REMOTE_ADDR")
BYTE_LOCKOUT_TIERS = ((5, 5), (10, 30), (20, 24 * 60))
BYTE_LOGIN_CAPTCHA_AFTER_FAILURES = env.int("BYTE_LOGIN_CAPTCHA_AFTER_FAILURES",
↪  default=3)
```

1. El comentario indica que el número de *proxies* debe ser 1 detrás de Render, pero el Blueprint (`render.yaml`) no fija `AXES_IPWARE_PROXY_COUNT`. Detrás de Cloudflare y Render la aplicación no obtiene la IP real del cliente, por lo que los fallos no se acumulan en el mismo par usuario–IP ni en la misma IP.

2. El bloqueo es solo por par usuario–IP: un atacante que rota usuarios nunca llega a 5 fallos por par.

3. Turnstile solo se activa tras 3 fallos de la misma IP, cuenta que también depende de la IP mal resuelta.

4. Agravante: en producción, Turnstile usa las llaves públicas de prueba de Cloudflare, que siempre aprueban. El widget lo advierte (Figura 28) y `check --deploy` lo avisaba con `byte.W003`. Las llaves reales quedaron pendientes en la guía de despliegue.

Recomendación.

1. Resolver la IP del cliente de forma confiable: fijar `AXES_IPWARE_PROXY_COUNT` según la cadena real de *proxies* o leer `CF-Connecting-IP`, y comprobarlo en los registros de intentos de axes.

2. Agregar un bloqueo por IP además del par usuario–IP, y un límite de tasa en la vista de inicio de sesión.

3. Cargar las llaves reales de Turnstile en Render y mostrarlo siempre en el inicio de sesión.

4. Agregar una prueba que simule peticiones detrás del *proxy* y repetir la prueba de los 30 intentos.

Figura 28

Turnstile en producción con llaves de prueba



Nota. Recorte de la captura del evaluador del restablecimiento de contraseña (evidencia `41cf5489…`). El widget dice «Solo para pruebas. Si se ve, informe al propietario del sitio».

PT-04. Secreto TOTP público en las cuentas demo del personal

Severidad: media (CVSS 3.1 estimado 4,8; AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N). **OWASP:** A07:2025 Fallas de autenticación.

Descripción. El comando `seed_demo_users` configura el segundo factor de todos los usuarios del personal con un secreto fijo, declarado en el código como valor público de desarrollo. El despliegue lo ejecuta en producción con `--force` a través de `bootstrap_demo`. Las cuentas demo del vendedor al superadministrador tienen, por tanto, un segundo factor que cualquiera puede calcular a partir del repositorio público. El evaluador lo usó en las pruebas autenticadas.

Impacto. El MFA de esas cuentas se reduce a la contraseña demo. Esa contraseña la genera Render y no está publicada, por lo que la complejidad de ataque es alta. Pero es la misma para todos los usuarios demo: quien la reciba para revisar la tienda obtiene acceso de superadministrador.

Recomendación. No sembrar secretos TOTP fijos fuera de desarrollo. En producción, crear las cuentas del personal sin autenticador y dejar que el *middleware* les exija activarlo en el primer acceso, o generar un secreto aleatorio por cuenta. Desactivar las cuentas demo con privilegios cuando no se estén usando.

PT-02. Panel del personal descubrible

Severidad: informativa. **OWASP:** A02:2025 Configuración de seguridad incorrecta. La ruta `/panel/` se descubre con una búsqueda de directorios básica. Está protegida por autenticación, permisos por rol y MFA del personal, y el administrador de Django sí está oculto. Se acepta el riesgo; si se quisiera reducir la exposición, se podría restringir el panel por IP o con un *proxy* de identidad.

PT-03. Filtro `|safe` sobre `help_text`

Severidad: informativa. **OWASP:** A05:2025 Inyección. Las plantillas `templates/partials/form_fields.html` y `templates/allauth/elements/fields.html` muestran `{{ field.help_text|safe }}`. Hoy los textos de ayuda los escribe el equipo, así que no es explotable, pero contradice la regla del proyecto que prohíbe `|safe` y se convertiría en XSS si algún texto de ayuda llegara a incluir datos del usuario. Ni semgrep ni las pruebas lo detectaron. Recomendación: quitar `|safe` y, si un texto necesita HTML, construirlo con `format_html` en el formulario.

Controles confirmados

La Tabla 14 resume el resultado por categoría del OWASP Top 10:2025, con su equivalencia en la edición 2021 que usó el evaluador.

Tabla 14

Resultado de la prueba por categoría del OWASP Top 10

Categoría 2025 (2021)	Resultado	Evidencia principal
A01 Control de acceso (A01, A10)	Sin hallazgos	Cliente en `/panel/` → 403; UUID ajeno → 404; CSRF activo; sin funciones que descarguen URL (SSRF).
A02 Configuración incorrecta (A05)	PT-02	Sin modo de depuración, cabeceras fuertes, CSP con *nonce*, sin CORS permisivo, admin oculto.
A03 Cadena de suministro (A06)	Sin hallazgos	pip-audit sin vulnerabilidades; JavaScript local con integridad verificada.
A04 Criptografía (A02)	Sin hallazgos	HTTPS con HSTS *preload*; cookies `__Host-` seguras, HttpOnly y SameSite.
A05 Inyección (A03)	PT-03	SQLi bloqueada por el WAF y el ORM; XSS escapado.
A06 Diseño inseguro (A04)	No evaluado	Requiere pruebas de lógica de negocio; la separación de funciones no se probó.
A07 Autenticación (A07)	PT-01, PT-04	Positivos: error genérico sin enumeración, MFA del personal, política de contraseñas con verificación de filtraciones.
A08 Integridad (A08)	Sin hallazgos	No se observaron deserialización insegura ni actualizaciones sin firma; los webhooks de pago no se probaron.
A09 Registro y alertas (A09)	No evaluado	No observable desde fuera.
A10 Condiciones excepcionales (—)	Sin hallazgos	Páginas de error propias, sin trazas.

Nota. Elaboración propia a partir de la evidencia `a909ca55…` (clasificación del evaluador, OWASP 2021), reorganizada según la edición 2025. Una flecha (→) indica la respuesta obtenida.

Limitaciones

1. La prueba de fuerza bruta se detuvo en 30 intentos; no se midió si Cloudflare aplica un límite a volúmenes mayores.

2. El inicio de sesión del personal con TOTP desde *scripts* terminó en 403 por la rotación del *token* CSRF, que es un control correcto. Por eso no se probaron la separación de funciones ni las operaciones del panel con sesión de personal.

3. No se probaron los pagos ni los webhooks contra la instancia, que usa el simulador.

4. La herramienta registró PT-01, PT-02 y PT-03 como observaciones verificadas manualmente, no por su flujo de repetición automática; por eso su reporte cuenta cero

hallazgos «verificados».

5. El evaluador clasificó primero PT-01 como baja y lo subió a media al encontrar la causa raíz en el código.

6. La re-prueba cubrió PT-01; PT-04 se verificó con pruebas automatizadas, no con un inicio de sesión del personal en producción.

Remediación y re-prueba

Los hallazgos de severidad media se corrigieron el mismo 13 de noviembre (commit `3d010e5`), con el mismo método del resto del proyecto: cambio con pruebas, verificaciones automáticas y despliegue.

PT-01: la causa raíz era más profunda

Al corregir PT-01 apareció una causa más profunda que la del informe: **django-ipware no estaba instalado**, así que los ajustes `AXES_IPWARE_*` nunca tuvieron efecto. django-axes, el límite de tasa (django-ratelimit), los límites de allauth y la auditoría usaban `REMOTE_ADDR`, que en Render es una IP privada del balanceador (`10.x`), compartida y cambiante. Fijar el número de *proxies*, como recomendaba el informe, no habría servido.

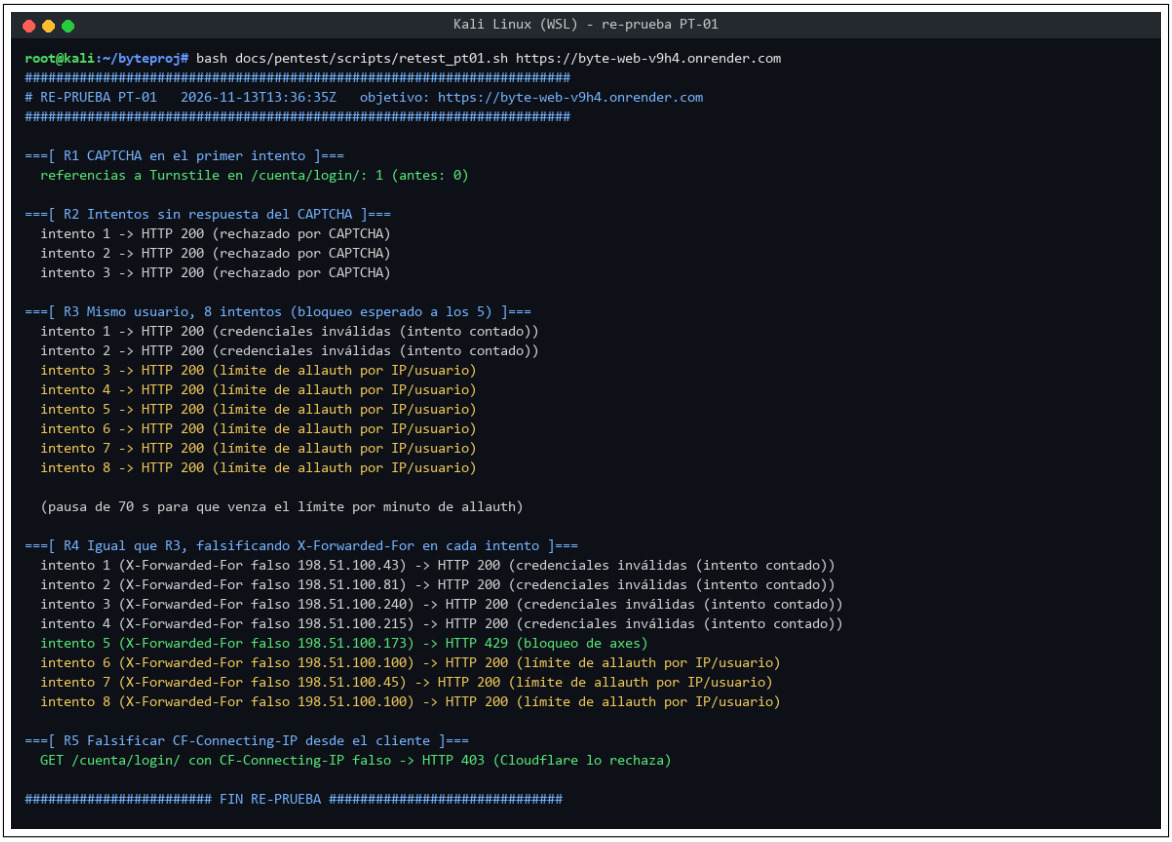
La corrección (ADR-0018) fue una sola función, `apps.core.net.client_ip`, que usan todos esos controles. En Render lee `CF-Connecting-IP`, que escribe Cloudflare; nunca la primera entrada de `X-Forwarded-For`, que Render no limpia y el cliente puede falsificar. Se agregaron además: CAPTCHA en todo inicio de sesión en producción, el chequeo `byte.W005` en `check --deploy` si la IP sale de `REMOTE_ADDR`, y una prueba que simula intentos detrás de *proxies* que rotan.

La re-prueba se ejecutó desde Kali contra producción (Figura 29) con cuentas inexistentes. El inicio de sesión ya muestra Turnstile en el primer intento y rechaza los envíos sin respuesta del CAPTCHA. Con el mismo usuario, cuatro intentos cuentan y el quinto recibe el bloqueo de axes (HTTP 429), aunque cada intento falsifique una IP distinta en `X-Forwarded-For`; el límite de allauth por IP rechaza los siguientes. Cloudflare responde 403 si el cliente intenta

enviar su propio `CF-Connecting-IP`. Antes de la corrección, 30 intentos no producían ningún bloqueo.

Figura 29

Re-prueba de PT-01 desde Kali contra producción



Nota. Salida real del script `docs/pentest/scripts/retest_pt01.sh`, ejecutado desde Kali Linux en WSL el 13 de noviembre de 2026 a las 13:36 UTC (registro en `docs/pentest/logs/retest-pt01.txt`), presentada como imagen de terminal. En R3, el límite por minuto de allauth seguía activo por una ejecución anterior; R4 muestra el comportamiento completo.

Queda un pendiente que no se puede resolver desde el código: Turnstile sigue con las llaves públicas de prueba. La re-prueba lo confirma, porque los intentos con el token de prueba de Cloudflare superan el CAPTCHA y llegan a contar como fallos. Hasta que se carguen las llaves reales en Render (paso 3 de `docs/despliegue.md`), la protección efectiva es el bloqueo y el límite por IP, no el CAPTCHA.

PT-04

El secreto TOTP público ahora solo se siembra en desarrollo (`DEBUG=True`). En cualquier otro entorno, el comando retira el secreto que hubiera dejado una carga anterior, con sus códigos de recuperación, y lo registra en la auditoría. El *middleware* del personal obliga a cada cuenta a activar su propio autenticador en el primer ingreso. Cuatro pruebas nuevas lo cubren, y el pre-despliegue lo aplica en cada despliegue de Render.

Verificaciones de la corrección

La suite pasó con 746 pruebas y 93 % de cobertura, y ruff, black, mypy y bandit quedaron sin hallazgos. semgrep no reportó nada en 417 archivos, y `check --deploy` pasó sin advertencias con la configuración de producción. La Tabla 15 resume el estado final.

Tabla 15

Estado de los hallazgos después de la remediación

ID	Estado	Evidencia o acción pendiente
PT-01	Corregido y re-probado	Bloqueo a los 5 intentos y límite por IP, aun falsificando `X-Forwarded-For`. Pendiente: cargar las llaves reales de Turnstile en Render.
PT-04	Corregido	Pruebas automatizadas; el pre-despliegue retira el secreto público en cada despliegue.
PT-02	Riesgo aceptado	Panel protegido por autenticación, permisos y MFA.
PT-03	Abierto (informativo)	Sin impacto actual; quitar `|safe` en una próxima iteración.

Nota. Elaboración propia a partir de la re-prueba y del commit `3d010e5`.

Lección para replicar

Las verificaciones automáticas de las fases anteriores (727 pruebas, semgrep sin hallazgos, ZAP sin fallos) no detectaron PT-01 ni PT-04, porque ambos dependen de cómo se desplegó la aplicación: un *proxy* delante, una configuración para una librería que ni siquiera estaba instalada y un comando de demostración ejecutado en producción. Pruebe siempre el sistema desplegado, no solo el código; agregue pruebas que simulen el entorno real (aquí, los *proxies*), y convierta cada supuesto de producción en un chequeo que avise antes de desplegar.

Conclusiones

El caso Byte muestra que el *vibe coding* puede producir una aplicación web de tamaño y exigencia reales cuando parte de un prompt maestro bien construido y no de una conversación improvisada. La persona no escribió el código, pero sí decidió el alcance, fijó las reglas, revisó cada entrega y exigió evidencias; la IA escribió, probó, verificó y documentó.

El factor que más pesó en el resultado no fue la capacidad del modelo, sino la calidad de la especificación y la disciplina de verificación que esa especificación imponía: cada fase cerró con las mismas comprobaciones, y los errores, incluidos los de la propia IA, quedaron registrados y corregidos. Ese registro es, a la vez, la mejor defensa ante una revisión y lo que permitió explicar con precisión los hallazgos de la prueba de penetración.

La prueba de penetración confirmó que los controles exigidos por el prompt funcionan donde dependen del código: control de acceso, protección contra inyección, CSRF, cabeceras y MFA del personal. Las fallas aparecieron en la frontera entre el código y el despliegue: una IP del cliente mal resuelta detrás del *proxy*, unas llaves de CAPTCHA de prueba y un comando de demostración ejecutado en producción. La conclusión práctica para quien construya con *vibe coding* es que la especificación debe exigir también la verificación del sistema desplegado, y que la prueba de penetración no es un trámite final, sino la última puerta de revisión del método. Cerrar el ciclo también fue parte del método: los dos hallazgos de severidad media se corrigieron con pruebas y se desplegaron el mismo día, y PT-01 se re-probó desde Kali contra producción.

Por último, este documento se redactó siguiendo las normas APA en su séptima edición (American Psychological Association, 2020), con apoyo en la guía de citas y referencias de Menéndez Echavarría (2020).

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.ª ed.). https://doi.org/10.1037/0000165-000

Anthropic. (s. f.). *Claude Code overview*. Claude Docs. https://docs.claude.com/en/docs/claude-code/overview

Astral. (s. f.). *uv*. https://docs.astral.sh/uv/

Congreso de Colombia. (2011). *Ley 1480 de 2011 por medio de la cual se expide el Estatuto del Consumidor y se dictan otras disposiciones*. http://www.secretariasenado.gov.co/senado/basedoc/ley_1480_2011.html

Congreso de Colombia. (2012). *Ley Estatutaria 1581 de 2012 por la cual se dictan disposiciones generales para la protección de datos personales*. http://www.secretariasenado.gov.co/senado/basedoc/ley_1581_2012.html

Django Software Foundation. (2025). *Django* (Ver. 5.2 LTS). https://www.djangoproject.com/

FIRST. (2019). *Common Vulnerability Scoring System v3.1: Specification document*. https://www.first.org/cvss/v3.1/specification-document

Karpathy, A. (2025, 2 de febrero). *There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists* [Publicación]. X. https://x.com/karpathy/status/1886192184808149383

Menéndez Echavarría, A. L. (2020). *Estilo de la American Psychological Association: citas y referencias [Normas APA, 7.ª edición]*. ResearchGate. https://doi.org/10.13140/RG.2.2.21676.41603

Microsoft. (s. f.). *Playwright*. https://playwright.dev/

OffSec. (s. f.). *Kali Linux* (Ver. Rolling). https://www.kali.org/

OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/

OWASP Foundation. (2025). *OWASP Top 10:2025*. https://owasp.org/Top10/2025/

PostgreSQL Global Development Group. (2024). *PostgreSQL* (Ver. 17). https://www.postgresql.org/

Render. (s. f.). *Blueprint YAML reference*. Render Docs. https://render.com/docs/blueprint-spec

Semgrep, Inc. (s. f.). *Semgrep*. https://semgrep.dev/

Wikimedia Commons. (s. f.). *Wikimedia Commons*. Wikimedia Foundation. https://commons.wikimedia.org/

Wompi. (s. f.). *Documentación para desarrolladores*. Wompi. https://docs.wompi.co/

ZAP Development Team. (s. f.). *ZAP* (Ver. 2.17.0). https://www.zaproxy.org/

Apéndice A

Prompt maestro completo

A continuación se transcribe, sin cambios, el prompt maestro con el que comenzó la construcción. Las tablas del original estaban separadas por tabulaciones.

```
0. Rol, contexto y reglas de trabajo

Actúa como arquitecto de software senior y desarrollador full-stack Django, con experiencia en
↪  e-commerce, ingeniería de datos y seguridad de aplicaciones (AppSec). Vas a construir
↪  desde cero una aplicación web de producción para Byte, una tienda de videojuegos, consolas
↪  y accesorios en Colombia (moneda COP, IVA 19 %, envíos nacionales por
↪  departamento/ciudad).

Reglas de trabajo obligatorias:

Trabaja por fases (sección 11). Al final de cada fase: ejecuta los tests, el linter y el
↪  análisis de seguridad, actualiza CHANGELOG.md, haz un commit con mensaje convencional
↪  (feat:, fix:, sec:, docs:…) y detente para que yo revise.
Antes de escribir código, crea CLAUDE.md con las convenciones del proyecto, los comandos
↪  (correr, testear, migrar, sembrar datos, desplegar) y las decisiones de arquitectura.
↪  Mantenlo actualizado.
Nunca pongas secretos en el código ni en el repo. Todo va por variables de entorno
↪  (.env.example documentado, .env en .gitignore).
Si una decisión tiene varias opciones razonables, elige la más simple y segura, y documéntala
↪  como ADR corto en docs/adr/.
Si una librería o servicio no está disponible o cambió, verifica la versión actual estable
↪  antes de fijarla y anota el cambio.
Código, nombres de modelos y comentarios en inglés; interfaz de usuario, mensajes y
↪  documentación en español.
Todo cambio de lógica de negocio importante debe tener tests.
1. Stack tecnológico
Capa    Tecnología
Backend    Python 3.12+, Django 5.2 LTS (o la LTS vigente), Django REST Framework
Frontend    Plantillas Django + HTMX + Alpine.js + Tailwind CSS (build con django-tailwind o
↪  CLI standalone), diseño responsive, modo oscuro, look "gamer" moderno
Base de datos    PostgreSQL 16+ (búsqueda full-text + pg_trgm)
Caché / colas    Redis + Celery + Celery Beat
Autenticación    django-allauth (incluye MFA/TOTP), django-axes (bloqueo por fuerza bruta)
Permisos    Grupos/permisos de Django + permisos a nivel de objeto (django-guardian o servicio
↪  propio)
Archivos    django-storages (S3-compatible, p. ej. Cloudflare R2) en producción; disco local
↪  en desarrollo
Big Data / BI    Polars + DuckDB + Parquet (data lake), scikit-learn, statsmodels, mlxtend,
↪  gráficas con Apache ECharts o Plotly
Procesamiento distribuido (opcional/local)    PySpark en contenedor Docker para el notebook de
↪  procesamiento masivo
Pagos    Pasarela colombiana en modo sandbox (Wompi o Mercado Pago) detrás de una interfaz
↪  PaymentGateway con implementación FakeGateway para desarrollo y tests
Correo    SMTP / servicio transaccional vía django-anymail; consola en desarrollo
Servidor    Gunicorn + WhiteNoise para estáticos
Calidad    ruff, black, mypy (gradual), pytest-django, factory_boy, coverage (≥ 80 %)
```

```
Seguridad en CI    bandit, pip-audit, semgrep, OWASP ZAP baseline, cyclonedx-py (SBOM)
Contenedores    Dockerfile multi-stage (usuario no root) + docker-compose.yml (web, worker,
↪  beat, postgres, redis)
Dependencias    uv o pip-tools con lockfile y hashes
2. Arquitectura y estructura del repositorio

Monolito modular Django con capa de servicios (la lógica de negocio NO vive en las vistas):

/config                 # settings/base.py, dev.py, prod.py, test.py; urls; celery.py;
↪  wsgi/asgi
/apps
  /accounts             # usuarios, perfiles, direcciones, MFA, roles
  /catalog              # plataformas, marcas, categorías, productos, variantes, atributos,
  ↪  bundles
  /pricing              # reglas de precio por configuración, historial de precios,
  ↪  promociones
  /inventory            # bodegas/sedes, stock, movimientos (kardex), reservas, alertas
  /purchasing           # proveedores, órdenes de compra, recepciones
  /cart                 # carrito (sesión + usuario), lista de deseos, comparador
  /orders               # checkout, pedidos, estados, facturas, devoluciones (RMA)
  /payments             # interfaz de pasarela, webhooks firmados, idempotencia
  /shipping             # zonas, tarifas, guías, seguimiento
  /promotions           # cupones, descuentos, flash sales, puntos de fidelidad
  /tradein              # retoma/compra de consolas y juegos usados
  /services_desk        # servicio técnico: tickets de reparación y garantías
  /reviews              # reseñas moderadas, preguntas y respuestas
  /pos                  # punto de venta en mostrador
  /notifications        # email, notificaciones in-app, alertas de stock/precio
  /cms                  # banners, páginas estáticas, noticias/lanzamientos
  /audit                # bitácora de auditoría y eventos de seguridad
  /analytics            # módulo Big Data / BI (ETL, data lake, modelos, dashboards)
  /api                  # DRF v1 (versionada)
/templates  /static  /docs  /scripts  /tests  /notebooks
Cada app: models.py, services.py (lógica), selectors.py (consultas), forms.py, views.py,
↪  permissions.py, tests/.
Identificadores públicos en URLs y API: UUID o slug, nunca el ID autoincremental.
Precios en DecimalField (o enteros en pesos), nunca float.
Genera el diagrama entidad-relación en Mermaid en docs/modelo-datos.md.
3. Usuarios, roles y control de acceso
3.1 Roles
Rol    Descripción
Invitado    Navega catálogo, usa carrito de sesión
Cliente    Compra, gestiona su cuenta, pedidos, retomas, tickets, reseñas
Cliente VIP    Cliente con nivel de fidelidad alto (precios/promos exclusivas) — es un
↪  atributo, no un rol administrativo
Vendedor / Cajero    POS, consulta de stock, creación de pedidos en tienda, aplicar cupones
↪  permitidos
Bodeguero    Inventario, movimientos, recepciones, traslados entre sedes
Técnico de soporte    Tickets de reparación, garantías, evaluación de retomas
Gerente de tienda    Todo lo operativo de su(s) sede(s), aprobaciones (descuentos altos,
↪  devoluciones, ajustes de inventario), reportes
Analista de datos    Solo lectura sobre el módulo de analítica; exportaciones anonimizadas;
↪  sin acceso a PII completa
Administrador    Gestión completa de la tienda y usuarios internos (excepto superadmins)
Superadmin    Configuración del sistema, roles y permisos, bitácora de seguridad
```

```
3.2 Requisitos de control de acceso
Denegar por defecto: toda vista del panel exige autenticación + permiso explícito. Crea
↪  mixins/decoradores propios (RoleRequiredMixin, ObjectPermissionMixin) y úsalos en TODAS
↪  las vistas y endpoints.
Permisos a nivel de objeto: un cliente solo ve sus pedidos, direcciones, tickets y retomas; un
↪  gerente solo ve las sedes asignadas.
Separación de funciones: quien crea un ajuste de inventario o un descuento > [X] % no puede
↪  aprobarlo.
Matriz rol × permiso generada automáticamente en docs/matriz-permisos.md a partir del código
↪  (comando de gestión export_permissions_matrix).
Comando setup_roles idempotente que crea grupos y permisos.
Tests que prueban, para cada endpoint sensible, que los roles sin permiso reciben 403/404
↪  (incluye pruebas de IDOR cambiando UUIDs).
El panel de administración personalizado vive en /panel/. El django.contrib.admin queda solo
↪  para superadmin, en una ruta no predecible configurable por variable de entorno y con MFA
↪  obligatorio.
4. Módulo Tienda (cliente final)
4.1 Catálogo y configuraciones de consolas
Tipos de producto: consolas, videojuegos (físicos y digitales/códigos), accesorios,
↪  coleccionables, tarjetas de regalo, servicios (garantía extendida, instalación).
Variantes con atributos configurables que cambian precio, SKU, stock e imágenes:
Modelo/versión (p. ej. estándar, slim, digital, pro, OLED, lite…)
Almacenamiento (GB/TB)
Color / edición (estándar, edición especial, bundle con juego)
Condición: nuevo, usado, reacondicionado (con grado A/B/C y garantía distinta)
Región / tipo de enchufe
Precio según configuración: precio base + modificadores por atributo, o precio fijo por
↪  variante; el selector de la ficha de producto actualiza precio, disponibilidad, SKU e
↪  imagen en vivo (HTMX), y deshabilita combinaciones que no existen.
Bundles / "arma tu setup": consola + juegos + accesorios con descuento por combo y validación
↪  de compatibilidad.
Verificador de compatibilidad: cada accesorio/juego declara plataformas compatibles;
↪  advertencia en carrito si algo no es compatible.
Ficha de videojuego: plataforma, género, clasificación ESRB/PEGI, idiomas, multijugador,
↪  desarrollador, fecha de lanzamiento, formato.
Preventas con fecha de lanzamiento, reserva con abono y notificación al llegar.
Historial de precios visible (gráfica pequeña en la ficha) y etiqueta "precio más bajo en 30
↪  días".
4.2 Descubrimiento
Búsqueda full-text con tolerancia a errores (pg_trgm) y autocompletado.
Filtros facetados: plataforma, precio, condición, marca, género, clasificación,
↪  disponibilidad, almacenamiento.
Orden: relevancia, precio, novedad, más vendidos, mejor calificados.
Comparador de hasta 4 productos (tabla de especificaciones).
Recomendaciones: "comprados juntos frecuentemente" y "también te puede gustar" alimentadas por
↪  el módulo de analítica (sección 6).
Vistos recientemente, lanzamientos, más vendidos, ofertas flash con contador.
4.3 Carrito y checkout
Carrito de invitado que se fusiona al iniciar sesión; reserva temporal de stock (TTL
↪  configurable).
Lista de deseos con alerta de bajada de precio y alerta de reposición de stock.
Checkout en pasos: dirección → envío (tarifa por zona o recoger en tienda) → pago →
↪  confirmación.
El servidor recalcula siempre precios, descuentos, impuestos y envío; jamás confía en valores
↪  del cliente.
```

```
Cupones (porcentaje, valor fijo, envío gratis, por categoría, primer pedido, uso único por
↪  usuario, vigencia, tope).
Puntos de fidelidad: acumulación por compra y redención en checkout.
Pagos con pasarela en sandbox; webhooks con verificación de firma e idempotencia; nunca
↪  almacenar datos de tarjeta (solo tokens/referencias de la pasarela).
Factura/recibo en PDF descargable (factura electrónica simulada con los campos que exige la
↪  DIAN, marcada como "documento de prueba").
4.4 Cuenta de cliente
Registro con verificación de correo, login, MFA opcional (TOTP), recuperación de contraseña
↪  segura.
Perfil, direcciones múltiples, preferencias de notificación, consentimiento de tratamiento de
↪  datos (Ley 1581 de 2012, Habeas Data) con fecha y versión de la política aceptada.
Historial de pedidos con seguimiento por estados (pendiente de pago → pagado → en preparación
↪  → enviado → entregado / cancelado / devuelto) y línea de tiempo.
Solicitud de devolución/RMA dentro de la ventana permitida.
Retoma (trade-in): el cliente cotiza su consola/juego usado (modelo, estado, accesorios
↪  incluidos) → obtiene oferta estimada → agenda revisión → el técnico confirma → crédito en
↪  tienda o pago.
Servicio técnico: abrir ticket de reparación o garantía, adjuntar fotos (validadas), seguir
↪  estado.
Reseñas con calificación (solo compradores verificados), preguntas y respuestas en producto.
Exportar y eliminar mis datos (derecho de supresión con anonimización de pedidos históricos).
5. Panel de administración (/panel/)

Interfaz propia (no el admin de Django) con menú según el rol del usuario.

Dashboard operativo: ventas del día/semana/mes, ticket promedio, pedidos por estado, productos
↪  con stock bajo, tickets abiertos, retomas pendientes, alertas de seguridad (solo
↪  superadmin).
Catálogo: CRUD de productos, variantes, atributos, plataformas, marcas, categorías, bundles;
↪  carga masiva por CSV/Excel con validación y vista previa de errores; galería de imágenes
↪  con optimización automática (WebP, miniaturas).
Precios y promociones: reglas de precio por configuración, cambios masivos por porcentaje,
↪  programación de precios futuros, flash sales, cupones, niveles de fidelidad.
Inventario: multi-sede/bodega, stock disponible/reservado/en tránsito, kardex de movimientos
↪  (entrada, salida, ajuste, traslado, devolución) con usuario y motivo, stock mínimo y punto
↪  de reorden, conteos cíclicos, ajustes con aprobación, control de seriales/IMEI para
↪  consolas.
Compras: proveedores, órdenes de compra, recepción parcial/total que actualiza inventario y
↪  costo promedio.
Ventas y pedidos: listado con filtros, detalle, cambio de estado con reglas de transición
↪  válidas, notas internas, reembolsos, devoluciones, impresión de guía/factura.
POS (punto de venta): venta rápida en mostrador con búsqueda/lector de código de barras,
↪  cliente opcional, varios medios de pago, apertura y cierre de caja con arqueo.
Clientes (CRM): ficha 360° (pedidos, valor de vida, segmento RFM, tickets, retomas),
↪  etiquetas, notas.
Retomas y servicio técnico: bandeja de evaluaciones y reparaciones, asignación a técnicos,
↪  estados, costos, garantías.
Moderación: reseñas y preguntas (aprobar, rechazar, responder).
Contenido: banners del home, páginas, noticias de lanzamientos.
Usuarios internos y roles: alta/baja de empleados, asignación de rol y sede, forzar MFA,
↪  revocar sesiones.
Reportes: ventas, inventario valorizado, rotación, márgenes, comisiones de vendedores;
↪  exportación CSV/Excel/PDF (con protección contra inyección de fórmulas en CSV).
```

```
Auditoría: quién hizo qué, cuándo, desde qué IP, valor anterior y nuevo, en todas las
↪  operaciones del panel.
Configuración: datos de la empresa, impuestos, zonas de envío, sedes, plantillas de correo.
6. Módulo Big Data / Inteligencia de Negocio (/panel/analytics/)

Objetivo: que la gerencia tome decisiones basadas en datos. Debe funcionar con volumen
↪  realista, no con 20 registros.

6.1 Generación de datos
Comando seed_bigdata --orders 500000 --customers 50000 --years 3 que use Faker (locale es_CO)
↪  para generar datos sintéticos coherentes: estacionalidad (diciembre, Black Friday,
↪  lanzamientos, prima de junio), ciudades colombianas, mezcla de canales (web/POS),
↪  devoluciones, retomas, visitas y eventos de navegación.
Inserción por lotes (bulk_create/COPY) para que sea rápida.
6.2 Pipeline de datos (arquitectura medallón)
Captura de eventos: modelo Event (vista de producto, búsqueda, agregar al carrito, inicio de
↪  checkout, compra) registrado de forma asíncrona y sin PII cruda (ID de usuario
↪  seudonimizado).
Bronze: extracción incremental desde PostgreSQL a archivos Parquet particionados por fecha.
Silver: limpieza, deduplicación y tipado con Polars.
Gold: modelo estrella (fact_sales, fact_inventory_snapshot, fact_events, dim_product,
↪  dim_customer, dim_date, dim_store, dim_channel) consultado con DuckDB.
Orquestado con Celery Beat (ejecución nocturna + botón "recalcular ahora" para
↪  analista/gerente), con registro de cada corrida (duración, filas, errores) y validaciones
↪  de calidad de datos (nulos, duplicados, rangos).
Notebook notebooks/procesamiento_spark.ipynb que hace el mismo procesamiento con PySpark sobre
↪  los Parquet, para demostrar escalamiento horizontal (corre en local con Docker; no se
↪  despliega).
6.3 Analítica y modelos
Análisis    Pregunta de negocio que responde
KPIs y tendencias (ventas, margen, ticket promedio, unidades, conversión)    ¿Cómo va el
↪  negocio y hacia dónde?
Pronóstico de demanda por producto/categoría (statsmodels / Holt-Winters o SARIMA)    ¿Cuánto
↪  debo comprar y cuándo?
Punto de reorden sugerido y riesgo de quiebre de stock    ¿Qué se va a agotar?
Análisis ABC / rotación de inventario y stock muerto    ¿Qué productos inmovilizan capital?
Segmentación RFM + K-Means de clientes    ¿A quién dirigir cada campaña?
Análisis de canasta (Apriori / reglas de asociación)    ¿Qué bundles armar? Alimenta
↪  "comprados juntos"
Elasticidad de precio por categoría    ¿Qué pasa si subo/bajo precios?
Embudo de conversión y abandono de carrito    ¿Dónde pierdo ventas?
Cohortes y retención, valor de vida del cliente (CLV)    ¿Los clientes vuelven?
Probabilidad de abandono (churn) — clasificación con scikit-learn    ¿Qué clientes estoy por
↪  perder?
Detección de anomalías en ventas y devoluciones    ¿Hay fraude o errores?
Rendimiento por sede, vendedor, canal y ciudad (mapa de Colombia)    ¿Qué sede o canal rinde
↪  más?
Valor de retomas vs reventa de usados    ¿Es rentable el programa de retoma?
6.4 Visualización y toma de decisiones
Dashboards interactivos con filtros globales (rango de fechas, sede, categoría, plataforma,
↪  canal) y comparación contra periodo anterior.
Cada tablero muestra un bloque "Insights y acciones recomendadas" generado por reglas (p. ej.
↪  "El producto X se agotará en ~9 días según el pronóstico; sugerido pedir 40 unidades").
Constructor simple de reportes (elegir métricas y dimensiones) y exportación.
```

```
Las métricas pesadas se calculan en segundo plano y se cachean en Redis; el dashboard nunca
↪  bloquea la petición web.
Documenta en docs/big-data.md: arquitectura del pipeline (diagrama Mermaid), modelo estrella,
↪  descripción de cada modelo, métricas de evaluación (MAPE, silhouette, precisión/recall) y
↪  cómo interpretar cada tablero.
7. Seguridad — OWASP Top 10:2025 (obligatorio)

Genera docs/seguridad.md con una matriz de trazabilidad: categoría OWASP → riesgo en esta app
↪  → control implementado → archivo/código → test que lo verifica. Incluye también la
↪  equivalencia con la edición 2021 por si la rúbrica la pide.

A01:2025 — Pérdida de control de acceso (incluye SSRF)
Denegar por defecto, mixins de rol y permisos a nivel de objeto (sección 3).
UUIDs públicos; tests automáticos de IDOR y escalamiento horizontal/vertical.
CORS restringido (django-cors-headers solo para orígenes permitidos).
SSRF: si se importan imágenes por URL, validar esquema, resolver DNS y bloquear IPs
↪  privadas/locales; lista blanca de dominios.
Rate limiting en endpoints sensibles (django-ratelimit).
A02:2025 — Configuración de seguridad incorrecta
DEBUG=False, ALLOWED_HOSTS, CSRF_TRUSTED_ORIGINS por entorno; SECRET_KEY fuerte desde variable
↪  de entorno.
Cabeceras: HSTS (con preload), X-Content-Type-Options, Referrer-Policy, Permissions-Policy,
↪  X-Frame-Options: DENY, CSP estricta con nonces (CSP nativa de Django si la versión la
↪  trae, si no django-csp); sin scripts inline sin nonce.
Cookies Secure, HttpOnly, SameSite=Lax/Strict; SECURE_SSL_REDIRECT.
python manage.py check --deploy sin advertencias como paso de CI.
Páginas 400/403/404/500 propias; sin listados de directorios; admin de Django en ruta oculta.
A03:2025 — Fallas en la cadena de suministro de software
Dependencias fijadas con hashes; pip-audit y Dependabot/Renovate; SBOM CycloneDX en cada
↪  release.
GitHub Actions fijadas por SHA, permisos mínimos del GITHUB_TOKEN.
Imagen Docker base oficial y slim, escaneada (Trivy); librerías de frontend con versión fija y
↪  SRI si vienen de CDN (preferible servirlas localmente).
A04:2025 — Fallas criptográficas
Hash de contraseñas con Argon2 (argon2-cffi) como primer hasher.
HTTPS en todo; cifrado de campos sensibles (teléfono, documento) en reposo con una clave
↪  gestionada por entorno.
No almacenar datos de tarjetas; tokens de pasarela solamente.
Tokens de recuperación y verificación de un solo uso y con expiración.
A05:2025 — Inyección
Solo ORM; si hay SQL crudo, siempre parametrizado (y revisado por semgrep/bandit).
Autoescapado de plantillas; prohibido |safe y mark_safe sobre datos de usuario; reseñas
↪  sanitizadas con nh3.
Validación de todos los formularios y serializers en servidor (tipos, rangos, longitudes,
↪  listas blancas).
Subida de archivos: validar tamaño, extensión y tipo real (magic bytes), re-procesar imágenes
↪  con Pillow, nombres aleatorios, almacenar fuera del webroot o en bucket privado.
Exportaciones CSV/Excel: neutralizar celdas que empiezan por = + - @ (CSV injection).
Consultas DuckDB del módulo de analítica: nunca concatenar filtros del usuario; usar
↪  parámetros y listas blancas de columnas.
A06:2025 — Diseño inseguro
Modelo de amenazas (STRIDE) del checkout, pagos, retomas y panel en docs/modelo-amenazas.md.
Concurrencia de stock con select_for_update() y transacciones atómicas (sin sobreventa).
Límites de negocio: máximo de unidades por pedido en preventas/ofertas, un cupón por usuario,
↪  anti-abuso de cupones y puntos.
```

```
Flujos de aprobación para descuentos altos, reembolsos y ajustes de inventario.
A07:2025 — Fallas de autenticación
django-allauth con verificación de correo, MFA TOTP (obligatorio para todo el personal),
↪  códigos de recuperación.
django-axes: bloqueo progresivo por usuario+IP; CAPTCHA (Cloudflare Turnstile o hCaptcha) tras
↪  intentos fallidos y en registro.
Validadores de contraseña (longitud ≥ 12 para personal, comunes, similitud) y verificación
↪  contra contraseñas filtradas (API k-anonymity de Have I Been Pwned).
Rotación de sesión al iniciar sesión y al cambiar privilegios; expiración por inactividad en
↪  el panel; listado y cierre de sesiones activas.
Mensajes de error genéricos ("credenciales inválidas") para no enumerar usuarios.
A08:2025 — Fallas de integridad de software o datos
Webhooks de pago verificados por firma HMAC + marca de tiempo + idempotencia por ID de evento.
Celery con serializador JSON (nunca pickle); nada de pickle/yaml.load inseguro sobre datos
↪  externos.
Firmar datos que viajan por el cliente (django.core.signing) cuando sea necesario.
Despliegue solo desde la rama principal tras pasar CI.
A09:2025 — Fallas de registro y alertas de seguridad
App audit: registro de inicios de sesión (éxitos y fallos), cambios de rol/permisos,
↪  operaciones del panel, exportaciones de datos, cambios de precio y ajustes de inventario.
Logs estructurados JSON, sin contraseñas, tokens ni datos personales completos (enmascarar).
Alertas (correo/notificación al superadmin) ante: fuerza bruta, muchos 403 seguidos, cambio de
↪  rol a administrador, exportaciones masivas.
Integración opcional con Sentry (con depuración de PII).
A10:2025 — Manejo inadecuado de condiciones excepcionales
Manejo de errores centralizado: el usuario ve un mensaje genérico con ID de correlación; el
↪  detalle va solo al log.
Fallar de forma cerrada: si falla la verificación de permisos, la pasarela o el antifraude, se
↪  niega la operación.
Timeouts y reintentos con backoff en llamadas externas; transacciones atómicas para que un
↪  error a mitad de proceso no deje pedidos o stock inconsistentes.
Tests de casos límite: pago rechazado, webhook duplicado, stock agotado durante el checkout,
↪  archivo corrupto, entrada vacía o gigante.
Protección de formularios (transversal)
CSRF en todos los formularios y peticiones HTMX (token en cabecera X-CSRFToken).
Validación en cliente solo como ayuda; la validación real es en servidor.
Honeypot + CAPTCHA en registro, contacto y recuperación de contraseña.
Límite de tamaño de petición y de archivos (DATA_UPLOAD_MAX_MEMORY_SIZE,
↪  FILE_UPLOAD_MAX_MEMORY_SIZE).
Pruebas de seguridad
Suite tests/security/ con pruebas de acceso por rol, IDOR, CSRF, XSS almacenado en reseñas,
↪  inyección en búsqueda y filtros de analítica, subida de archivos maliciosos y rate
↪  limiting.
En CI: bandit, semgrep --config p/django, pip-audit, manage.py check --deploy, y OWASP ZAP
↪  baseline contra el entorno de staging o un contenedor levantado en el pipeline. El
↪  pipeline falla con hallazgos altos.
8. API REST (/api/v1/)
DRF con autenticación por sesión para el frontend y por token/JWT de corta duración para
↪  integraciones.
Endpoints de catálogo (público, solo lectura), carrito, pedidos del cliente, y endpoints
↪  internos de inventario/analítica protegidos por rol.
Paginación, throttling por usuario/IP, filtros con lista blanca de campos.
Documentación OpenAPI con drf-spectacular (Swagger UI solo para usuarios autenticados del
↪  personal en producción).
```

```
9. Calidad, rendimiento y experiencia
Tests: unitarios (servicios), integración (vistas/API), e2e básicos del flujo de compra
↪  (Playwright opcional). Cobertura ≥ 80 % en services.py.
Evitar N+1 (select_related/prefetch_related), índices en campos de filtro, caché de fragmentos
↪  del catálogo.
Accesibilidad básica (contraste, etiquetas, navegación con teclado), SEO (metadatos, sitemap,
↪  datos estructurados de producto).
Internacionalización preparada (gettext), formato de moneda COP.
Datos de demostración: comando seed_demo con catálogo realista (consolas actuales y retro,
↪  juegos, accesorios con sus variantes y precios), usuarios de prueba por cada rol
↪  (credenciales solo en README de desarrollo y nunca en producción).
10. Despliegue en [PLATAFORMA_DE_DESPLIEGUE] y disponibilidad

Plataforma por defecto: Render. Si uso otra (Railway, Fly.io, un VPS con Docker, etc.), adapta
↪  esta sección y documenta el cambio.

Archivo de infraestructura como código (render.yaml Blueprint o equivalente) con: servicio web
↪  (Gunicorn), worker de Celery, Celery Beat (o cron de la plataforma), PostgreSQL gestionado
↪  y Redis/Key-Value gestionado.
Antes de desplegar, verifica los límites actuales del plan elegido (gratuito o de pago): si el
↪  plan no admite worker en segundo plano, implementa un modo alternativo (tareas programadas
↪  vía cron job de la plataforma o ejecución síncrona controlada) y documéntalo.
Script de build: instalar dependencias desde lockfile, compilar Tailwind, collectstatic;
↪  comando de release: migrate y setup_roles.
Variables de entorno documentadas en .env.example y en docs/despliegue.md (sin valores
↪  reales).
Endpoint /healthz (base de datos + Redis) para el health check de la plataforma.
Archivos de usuario en bucket S3-compatible privado con URLs firmadas.
HTTPS forzado, dominio personalizado opcional.
CI/CD con GitHub Actions: lint → tests → seguridad → build de imagen → despliegue automático
↪  solo si todo pasa.
Respaldos: documentar la política de backups de la base de datos y un script de restauración.
Al final, despliega la aplicación, ejecuta seed_demo (y un seed_bigdata reducido acorde al
↪  plan) en producción, verifica manualmente el flujo completo (registro → compra → panel →
↪  analítica) y entrégame la URL pública y el usuario demo de cada rol.
11. Plan de fases (detente al final de cada una)
Fase    Entregable
1. Fundaciones    Estructura, settings por entorno, Docker Compose, CI básico, CLAUDE.md, ADRs
↪  iniciales, modelo de datos y diagrama ER
2. Usuarios y seguridad base    Auth con allauth + MFA, axes, roles y permisos, mixins,
↪  auditoría, cabeceras/CSP, tests de acceso
3. Catálogo y precios    Productos, variantes y configuraciones con precio dinámico, búsqueda,
↪  filtros, comparador, historial de precios
4. Carrito, checkout y pagos    Carrito, cupones, envíos, pasarela sandbox, webhooks firmados,
↪  pedidos, facturas PDF, correos
5. Panel administrativo    Dashboard, inventario multi-sede y kardex, compras, pedidos, POS,
↪  CRM, moderación, reportes
6. Módulos de valor agregado    Retomas, servicio técnico, fidelidad, preventas, alertas de
↪  precio/stock, devoluciones
7. Big Data / BI    Generador de datos, pipeline medallón, modelo estrella, modelos
↪  analíticos, dashboards con insights, notebook PySpark
8. Endurecimiento de seguridad    Matriz OWASP completa, modelo de amenazas, suite de
↪  seguridad, ZAP, SBOM, revisión de hallazgos
9. Despliegue    Infraestructura como código, CI/CD, despliegue, datos demo, verificación
↪  end-to-end
```

```
10. Documentación final    README, manual de usuario por rol (con capturas), manual técnico,
↪  docs/seguridad.md, docs/big-data.md, guion de sustentación
12. Criterios de aceptación finales
 La app está desplegada y accesible por HTTPS con una URL pública.
 Existe al menos un usuario demo por rol y cada rol solo ve lo que le corresponde.
 Un cliente puede configurar una consola (versión/almacenamiento/condición), ver el precio
 ↪  cambiar, comprar con la pasarela sandbox y seguir su pedido.
 El inventario se descuenta correctamente y no hay sobreventa bajo concurrencia (test que lo
 ↪  demuestre).
 El panel permite gestionar catálogo, inventario, ventas, compras, POS, clientes, retomas y
 ↪  reportes.
 El módulo de analítica procesa ≥ 500.000 pedidos en local y muestra pronóstico, RFM, canasta,
 ↪  embudo y recomendaciones accionables.
 manage.py check --deploy sin advertencias; bandit, pip-audit y ZAP sin hallazgos altos.
 docs/seguridad.md mapea las 10 categorías OWASP 2025 a controles y tests concretos.
 Cobertura de tests ≥ 80 % en la capa de servicios y CI en verde.
```

Apéndice B

Índice de evidencias

Todas las evidencias están versionadas en el repositorio https://github.com/NeySeb832/tienda_byte_hackingetico. La Tabla B1 indica dónde encontrar las de cada fase.

Tabla B1

Ubicación de las evidencias por fase

Fase	Bitácora	Capturas	Commit
1	`docs/bitacora/fase-01.md`	`docs/capturas/fase-01/`	`47dd77a`
2	`docs/bitacora/fase-02.md`	`docs/capturas/fase-02/`	`ca3cd9b`
3	`docs/bitacora/fase-03.md`	`docs/capturas/fase-03/`	`78ce40e`
4	`docs/bitacora/fase-04.md`	`docs/capturas/fase-04/`	`4f692c8`
5	`docs/bitacora/fase-05.md`	`docs/capturas/fase-05/`	`373a861`
6	`docs/bitacora/fase-06.md`	`docs/capturas/fase-06/`	`78c400b`
7	`docs/bitacora/fase-07.md`	`docs/capturas/fase-07/`	`6b81d3d`
8	`docs/bitacora/fase-08.md`	`docs/capturas/fase-08/`	`66a1c2d`
9	`docs/bitacora/fase-09.md`	`docs/capturas/fase-09/`	`015be4d`
10	`README.md`, `docs/manuales/`	—	`c464a1e`

Nota. Cada carpeta de capturas contiene la salida de texto de cada verificación (`NN-*.txt`), su imagen de terminal (`NN-*.svg`), el resumen (`00-resumen.svg`) y las capturas de pantalla.

Las evidencias de la prueba de penetración están en el paquete `pentest-byte-web-v9h4-owasp.zip`, que se entrega aparte. Contiene el reporte ejecutivo y el reporte en formato JSON (`reporte/`), los 45 artefactos de evidencia nombrados por su *hash* SHA-256 (`evidencia/`) y las capturas de navegador y de terminal (`capturas/`). En el informe, cada evidencia se cita por los primeros caracteres de su *hash*. El paquete no se publica en el repositorio porque las notas de las pruebas autenticadas incluyen el secreto TOTP de las cuentas demo (PT-04).